Numerical Linear Algebra and Modern High-Performance Computing

A Concise Course of Computational Physics
Volume 0

November 2026

Reading Routes

Spatial discretization produces finite-dimensional operators. Their structure constrains solvers, and the solvers' data dependencies constrain machine execution. This volume follows that chain through factorization, Krylov spaces, preconditioning, multigrid, and performance analysis. Volume I develops physical modeling and spatial discretization; Volume II develops field geometry, constraints, and structure-preserving discretization. Here these results become inputs to reliable and efficient computation.

The first three chapters establish matrix, residual, and error terminology. Parts II and III study factorization and projection; Part IV combines them through preconditioning and multilevel methods. Part V uses Roofline to connect operators, time integration, and parallel communication. A first pass may follow Chapters 1, 2, 3, 4, 10, 12, 16, 19, 22, 24, and 29 before returning to assumptions and variants.

Each chapter supplies objectives, conceptual connections, minimal examples, and specific AI expansion questions. Omitted proofs and implementation details have search terms. Scalars are real unless stated otherwise; replace transpose by conjugate transpose for complex problems. N usually counts algebraic unknowns; n counts grid points per direction. CPU experiments use double precision. Code, figures, and measurement records are shared with the Chinese edition under `Vol0_Linear_Algebra_HPC/`. Models, timings, and unexecuted experiments are identified separately.

Contents

Reading Routes ii

I From Discrete Equations to Matrix Systems 1

1 Why Scientific Computing Leads to Matrix Problems 2
1.1 From Derivatives to Five Unknowns . . . 2
1.2 Assembly, Blocks, and Nonlinearity . . . 3
1.3 Storing a Matrix or Applying an Operator . . . 3

2 Matrix Structure and Solver Difficulty 5
2.1 Energy Structure and Nullspaces . . . 5
2.2 Three Measures of Scale . . . 5
2.3 A Minimal Counterexample and Mesh Effects . . . 6

3 Residuals, Errors, and Numerical Convergence 7
3.1 Computable Quantities and Unknown Error . . . 7
3.2 Stopping and Finite Precision . . . 8
3.3 Experiments and Monitoring . . . 8

II Matrix Factorization Methods 9

4 Gaussian Elimination and LU Factorization 10
4.1 Elimination, Permutations, and Triangular Systems . . . 10
4.2 A Complete Four-by-Four Example . . . 10
4.3 Cost, Reuse, and Verification . . . 11

5 Sparse LU and Large Sparse Systems 12
5.1 Elimination Connects Neighbors . . . 12
5.2 Ordering and Two-Stage Analysis . . . 12
5.3 A Poisson Experiment . . . 13

6 Cholesky and LDL$^{\mathbf{T}}$ Factorizations 14
6.1 Cholesky Recurrences and an Example . . . 14
6.2 Indefinite Problems and Block Pivots . . . 14
6.3 Physical Nullspaces and Solver Strategy . . . 15

7 QR and Least-Squares Problems 16
7.1 Projection and Conditioning . . . 16
7.2 Three Orthogonalization Mechanisms . . . 16
7.3 A Minimal Example and Rank Deficiency . . . 17

8 Incomplete Factorization **18**
8.1 Truncated Elimination . . . 18
8.2 Minimal Examples . . . 18

9 LU-SGS and Approximate Factorization **20**
9.1 Approximate Factors and Two Triangular Solves . . . 20
9.2 Iteration, Preconditioning, and Pseudo-Time . . . 20
9.3 A Three-Point Check . . . 21

III Krylov Subspace Methods **22**

10 The Krylov Subspace Idea **23**
10.1 Generating a Search Space . . . 23
10.2 Projection and Minimal Polynomials . . . 24
10.3 A Two-by-Two Calculation . . . 24

11 Arnoldi and Hessenberg Decomposition **25**
11.1 The Arnoldi Recurrence . . . 25
11.2 Two Steps for the Four-by-Four Matrix . . . 25
11.3 Breakdown Diagnostics and Cost . . . 26

12 GMRES **27**
12.1 From a Large Residual to Small Least Squares . . . 27
12.2 Restart and Preconditioning Semantics . . . 27
12.3 The Computational Interface . . . 28

13 Conjugate Gradients **29**
13.1 Energy Minimization and Short Recurrences . . . 29
13.2 Two Steps and an Error Bound . . . 29
13.3 Conditions for PCG . . . 30

14 BiCG and BiCGSTAB **31**
14.1 Two-Sided Projection . . . 31
14.2 An Executable Recurrence . . . 31
14.3 A Minimal Experiment . . . 32

15 Choosing and Extending Krylov Methods **33**
15.1 Structure and Projection Choices . . . 33
15.2 Actual Directions in FGMRES . . . 33
15.3 Combinations and Minimal Examples . . . 34

IV Preconditioning and Multilevel Methods **35**

16 The Role and Cost of Preconditioning **36**
16.1 Transformations and Residuals . . . 36
16.2 A Minimal Example and Cost . . . 37

17 Stationary Iteration and Smoothing **38**
17.1 Matrix Splittings . . . 38
17.2 A Fourier Smoothing Example . . . 38
17.3 Implementation and Cost . . . 39

18 Incomplete Factorization and Domain Decomposition **40**
18.1 Combining Local Solves . . . 40
18.2 Coarse Spaces and Block Systems . . . 40
18.3 A Minimal Experiment . . . 41

19 Geometric Multigrid **42**
19.1 Error Equations and Coarse Correction . . . 42
19.2 One-Dimensional Interpolation and Recursion . . . 42
19.3 Experiments and Complexity . . . 43

20 Algebraic Multigrid **44**
20.1 From Geometric to Algebraic Smoothness . . . 44
20.2 Complexity and Reliable Combinations . . . 45
20.3 An Executable Teaching Model . . . 45

21 Solver Combinations and Engineering Selection **46**
21.1 Conditions for Combinations . . . 46
21.2 Tolerances and Nonlinear Outer Iteration . . . 46
21.3 A Minimal Selection Experiment . . . 47

V Roofline-Driven High-Performance Computing **48**

22 The Roofline Performance Model **49**
22.1 Two Resource Bounds . . . 49
22.2 GEMM and CSR-SpMV . . . 50
22.3 Models, Timing, and Calibration . . . 50

23 Computational and Memory Fingerprints of Discretizations **52**
23.1 Finite Differences and Finite Volumes . . . 52
23.2 Finite Elements, DG, and FR . . . 52
23.3 LBM Streaming and Collision . . . 53
23.4 A Common Comparison . . . 53

24 Yee-FDTD: Discrete Structure and Performance **55**
24.1 Degrees of Freedom, Materials, and the Grid . . . 55
24.2 Topology and Matrix-Free Updates . . . 56
24.3 Auditable Work and Traffic . . . 56
24.4 Stability, Dispersion, and Acceptance . . . 57

25 Time Integration and Total Solution Cost **58**
25.1 Method of Lines and Stability Regions . . . 58
25.2 Staggering, Symplecticity, and Phase . . . 58
25.3 ADER, CERK, and LTS . . . 59
25.4 IMEX and Stiff Sources . . . 59
25.5 Minimal Comparisons . . . 59

26 Memory Hierarchies and Operator Implementation **61**
26.1 Hierarchy, Locality, and Layout . . . 61
26.2 Fusion and Blocking . . . 61
26.3 SIMD and Minimal Experiments . . . 62

27 Parallel Throughput on CPUs and GPUs **63**
27.1 CPU Threads and Memory Placement . . . 63
27.2 GPU Execution Resources . . . 63
27.3 Transfers, Asynchrony, and Residency . . . 64
27.4 A Reproducible Interface . . . 64

28 Distributed Computing and Communication Cost **65**
28.1 Communication and Scaling . . . 65
28.2 Halos, Reductions, and Overlap . . . 65
28.3 A Minimal MPI Experiment . . . 66

29 Integrated Cases Across the Volumes **67**
29.1 Krylov and AMG: From Poisson to Preconditioning . . . 67
29.2 Complete Maxwell Evolution . . . 67
29.3 Complete High-Order DG and the FR Interface . . . 68
29.4 Access Comparisons: FDM, FVM, FEM, and LBM . . . 69
29.4.1 Regular and Staggered Stencils . . . 69
29.4.2 Adjacency, Irregular Access, and Geometry . . . 69
29.4.3 Local Matrices and Population Layout . . . 69
29.5 Common Acceptance and Selection . . . 70

A Reproduction and Measurement Conventions **71**
A.1 A Common Error and Cost Record . . . 71
A.2 Transferability . . . 71
A.3 Recorded Numerical Results . . . 71
A.4 GPU and MPI Records . . . 72

References **73**

Part I

From Discrete Equations to Matrix Systems

Chapter 1

Why Scientific Computing Leads to Matrix Problems

Chapter Overview

Volumes I and II construct finite-dimensional representations and discrete fluxes; the next task is to solve the resulting equations. Unknowns, local coupling, and boundary treatment establish the correspondence between a matrix and its discrete operator. This correspondence guides factorization and iterative methods and provides a common view of assembled and matrix-free implementations.

Learning Objectives Derive a matrix from one-dimensional Poisson; distinguish mass, stiffness, and Jacobian matrices; recognize the common operator behind assembled and matrix-free implementations.

Core Question How do local differential relations become couplings among finitely many unknowns?

1.1 From Derivatives to Five Unknowns

Consider $-u'' = f$ on $(0,1)$ with $u(0) = g_0, u(1) = g_1$. Set $h = 1/6$ and $u_i \approx u(ih)$. The five interior equations are

$$A\boldsymbol{u} = \boldsymbol{b}, \quad A = h^{-2}\begin{bmatrix} 2 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} f_1 + g_0/h^2 \\ f_2 \\ f_3 \\ f_4 \\ f_5 + g_1/h^2 \end{bmatrix}.$$

Rows represent equations and columns represent degrees of freedom. Each local difference reads three values, giving at most three nonzeros per row; boundary data enter the right-hand side with their coefficients. For $u = x(1-x)$ and $f = 2$, the second difference is exact, providing a direct check of the matrix, signs, and boundary treatment.

Potential Conceptual Jump

Potential Question: Why distinguish definitions of unknowns after obtaining a finite-dimensional system?
Author Supplement: Nodal values, cell averages, and basis coefficients are different coordinates. The discrete operator maps them to residuals; its entries encode trial spaces, testing, and neighboring interactions. Changing coordinates or boundary elimination changes the matrix even when the PDE is unchanged.
Core Conceptual Connection: Continuous operator $\longrightarrow$ degrees of freedom and testing $\longrightarrow$ algebraic coupling.

1.2 Assembly, Blocks, and Nonlinearity

The finite-element approximation $u_h = \sum_j u_j \phi_j$ gives

$$M_{ij} = \int \phi_i \phi_j, \qquad K_{ij} = \int \nabla \phi_i \cdot \nabla \phi_j, \qquad M\dot{\boldsymbol{u}} + K\boldsymbol{u} = \boldsymbol{f}.$$

Basis functions with overlapping support couple. Local matrices are assembled through local-to-global numbering. In finite volumes, one interface flux enters both neighboring residuals. DG volume terms form element blocks, while face fluxes form adjacency blocks. Multiphysics may produce block or saddle-point systems. Globally supported spectral bases can yield dense differentiation matrices. For a nonlinear residual $\boldsymbol{F}(\boldsymbol{u}) = 0$, the Newton correction satisfies

$$J(\boldsymbol{u}_k)\delta\boldsymbol{u} = -\boldsymbol{F}(\boldsymbol{u}_k), \qquad J_{ij} = \partial F_i / \partial u_j.$$

J linearizes the residual at the current state. Together with time-discrete mass terms and boundary treatment, it determines the linear system.

1.3 Storing a Matrix or Applying an Operator

Banded, block-sparse, Kronecker, and regular-stencil representations describe different structures at the same system size. A matrix-free implementation computes $\boldsymbol{v} \mapsto A\boldsymbol{v}$ directly, using element integration or neighboring differences without storing all entries. Preconditioning may still use an assembled approximation.

Cost and Applicability Tridiagonal multiplication takes $O(N)$ time and storage; dense storage takes $O(N^2)$. Matrix-free evaluation reduces entry and index reads but can repeat geometry, coefficient, or integration work. Compare implementations of the same operator at the same accuracy.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Connections Volume I constructs M, K, J through spatial discretization. Chapters 5, 23, and 26 discuss fill-in, operator fingerprints, and data layout.

Ask AI

Ask AI: For the five-interior-point problem, take $g_0 = 1, g_1 = 2, f = 0$. Derive each right-hand-side entry and verify the linear exact solution. Then construct the linear finite-element mass and stiffness matrices.
Follow-up: Implement the same difference in CSR and as a matrix-free function. Compare outputs and identify their shared boundary convention.

Further Detail and Search General weak boundary enforcement and nonlinear convergence proofs are omitted. See [1, 3].

Keywords: degrees of freedom; assembly; Jacobian; matrix-free operator

Chapter 2

Matrix Structure and Solver Difficulty

Chapter Overview

Once the discrete operator is identified, symmetry, definiteness, nonnormality, and sensitivity connect matrix structure to solution difficulty. These properties guide the choice of Cholesky, CG, GMRES, and preconditioning. The selection proceeds from algorithmic assumptions to problem sensitivity and observed convergence.

Learning Objectives Identify SPD and indefinite systems; distinguish eigenvalues, singular values, condition numbers, and spectral radii; interpret the limitations of nonnormal examples.

Core Question Why can equally sparse matrices require different solution strategies?

2.1 Energy Structure and Nullspaces

A real matrix is SPD if $A = A^T$ and $\boldsymbol{x}^T A\boldsymbol{x} > 0$ for $\boldsymbol{x} \neq 0$; nonnegativity alone gives positive semidefiniteness. Dirichlet Poisson matrices are SPD. Pure Neumann discretizations generally have a constant nullspace and require compatible data and a normalization. A symmetric indefinite quadratic form takes both signs, for example

$$A_s = \begin{bmatrix} 2 & 1 \\ 1 & 0 \end{bmatrix}.$$

A saddle-point matrix $\left[\begin{smallmatrix} K & B^T \\ B & 0 \end{smallmatrix}\right]$ is generally indefinite even when K is SPD. Upwind transport, nonsymmetric boundary treatment, or linearization can produce nonsymmetry. SPD makes $\frac{1}{2}\boldsymbol{x}^T A\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{x}$ strictly convex, gives positive Cholesky pivots, and supports CG energy minimization and positive denominators. These conclusions do not extend directly to nonsymmetric matrices.

2.2 Three Measures of Scale

For invertible A,

$$\kappa_2(A) = \|A\|_2 \left\|A^{-1}\right\|_2 = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}, \qquad \rho(A) = \max_i |\lambda_i(A)|.$$

Singular values control Euclidean norm amplification, eigenvalues describe invariant directions, and the spectral radius characterizes asymptotic convergence of a fixed iteration. For SPD matrices these quantities give $\kappa_2 = \lambda_{\max}/\lambda_{\min}$; general matrices require the singular-value ratio.

Potential Conceptual Jump

Potential Question: Why can a system be difficult when every eigenvalue equals 1?
Author Supplement: Nonnormal eigenvectors may be nearly dependent, or a complete eigenbasis may not exist. Norms of error-propagating matrix polynomials also depend on eigenvector conditioning and Jordan structure. Interpret spectra alongside singular values, nonnormality, and measured residual histories.
Core Conceptual Connection: Eigenvalue locations $\longrightarrow$ polynomial action $\longrightarrow$ vector-norm response.

2.3 A Minimal Counterexample and Mesh Effects

Take $A_K = \left[\begin{smallmatrix} 1 & K \\ 0 & 1 \end{smallmatrix}\right]$. Its eigenvalues remain 1, yet $A_K(0,1)^T = (K,1)^T$, and $\kappa_2(A_K)$ increases with K. Exact-arithmetic GMRES can still terminate in two steps, so conditioning alone does not predict iteration counts. The one-dimensional Poisson eigenvalues are

$$\lambda_j = 4h^{-2}\sin^2\frac{j\pi}{2(N+1)}, \quad 1 \le j \le N,$$

Thus $\kappa_2 = O(h^{-2})$ under refinement. Improved discretization accuracy can accompany a harder algebraic system.

Cost and Applicability A full singular-value decomposition is expensive for large sparse systems. Start from PDE structure, boundaries, and discrete energy; supplement these with small spectral estimates, residual monitoring, and mesh-sequence experiments.

Ask AI

Ask AI: Compute singular values and condition numbers of A_K for $K = 1, 10, 100$. Choose right-hand sides with different one- and two-step GMRES residuals.
Follow-up: Derive the Poisson eigenvalues and verify conditioning under mesh refinement. Explain why this does not specify the iteration growth for every right-hand side.

Connections These conditions support Chapters 6, 13, and 15; Chapter 16 examines preconditioned structure.

Further Detail and Search Pseudospectra and general nonnormal convergence bounds are omitted. See [2, 3].

Keywords: SPD; singular values; condition number; nonnormality; pseudospectrum

Chapter 3

Residuals, Errors, and Numerical Convergence

Chapter Overview

After identifying matrix properties, we need a stopping criterion. This chapter connects sensitivity, floating-point implementation, and iteration monitoring through the exact residual–error relation. Report both normalization and the true residual, and assess algebraic error against discretization error and the physical quantities of interest.

Learning Objectives Distinguish forward error, backward error, and residual; include absolute tolerances; recognize drift between recursive and true residuals.

Core Question What is established when software reports convergence?

3.1 Computable Quantities and Unknown Error

Define $\boldsymbol{e}_k = \boldsymbol{x}_* - \boldsymbol{x}_k$ and $\boldsymbol{r}_k = \boldsymbol{b} - A\boldsymbol{x}_k$. Then

$$A\boldsymbol{e}_k = \boldsymbol{r}_k, \quad \frac{\|\boldsymbol{e}_k\|}{\|\boldsymbol{x}_*\|} \le \kappa(A)\frac{\|\boldsymbol{r}_k\|}{\|\boldsymbol{b}\|} \quad (\boldsymbol{b} \neq 0).$$

A small residual means approximate satisfaction of the equations; the inverse operator also controls solution error. For $A = \mathrm{diag}(1, 10^{-8})$, $\boldsymbol{x}_* = (1, 1)^T$, and $\boldsymbol{x}_k = (1, 0)^T$, the relative residual is approximately 10^{-8}, while relative solution error is approximately $1/\sqrt{2}$. A normwise relative backward error is

$$\eta_k = \frac{\|\boldsymbol{b} - A\boldsymbol{x}_k\|}{\|A\|\,\|\boldsymbol{x}_k\| + \|\boldsymbol{b}\|}$$

This allows perturbations of A and $\boldsymbol{b}$ at a common relative scale that make $\boldsymbol{x}_k$ exact for the perturbed problem. Componentwise backward error instead normalizes with $|A||\boldsymbol{x}_k| + |\boldsymbol{b}|$ entry by entry.

Potential Conceptual Jump

Potential Question: Why not reduce every residual to machine precision?
Author Supplement: Discretization, data, and floating-point arithmetic each impose resolution limits. Oversolving wastes time without improving the target error; premature termination contaminates physical quantities. Allocate an error budget, then use conditioning, stability estimates, or quantity-of-interest tests to assess algebraic accuracy.
Core Conceptual Connection: Discretization error budget $\longrightarrow$ algebraic tolerance $\longrightarrow$ verifiable result.

3.2 Stopping and Finite Precision

A common true-residual criterion is

$$\|\boldsymbol{r}_k\| \leq \max(\texttt{atol}, \texttt{rtol}\,\|\boldsymbol{b}\|).$$

For $\boldsymbol{b} = 0$, use an absolute threshold or the initial residual scale; $A\boldsymbol{x} = 0$ still requires attention to nullspaces. When variable units differ substantially, define row and column scaling so that a single component does not dominate the norm. The model $\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta)$, $|\delta| \lesssim u$, describes a basic floating-point operation within the normal range. Cancellation, overflow, and underflow limit relative accuracy. Stability concerns how an algorithm handles perturbations; conditioning concerns the sensitivity of the problem. Rounding can cause the recursively maintained Krylov residual to drift from $\boldsymbol{b} - A\boldsymbol{x}_k$, so final verification recomputes the true residual. Small denominators, lost orthogonality, and preconditioner failure enter explicit breakdown or stagnation diagnostics that guide continuation or a change of strategy.

3.3 Experiments and Monitoring

For the diagonal example, print relative residual, relative error, and η_k. On Poisson mesh sequences, fix a physical error target and vary `rtol` to expose oversolving. Newton outer iterations also require nonlinear-residual monitoring: inner linear convergence does not establish overall convergence.

Cost and Applicability A true residual requires an operator application and a norm; distributed norms generally require a global reduction. Balance monitoring frequency and reliability, retaining a final acceptance check.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Compute forward error, relative residual, and backward error for the diagonal example. Explain why their values are consistent.
Follow-up: Design stopping logic for zero right-hand sides, scaling, and iteration limits. Compare recursive and recomputed residual costs.

Further Detail and Search Componentwise perturbation theory and residual-replacement error analysis are omitted. See [1, 4].

Keywords: forward error; backward error; true residual; residual replacement; inexact Newton

Part II

Matrix Factorization Methods

Chapter 4

Gaussian Elimination and LU Factorization

Chapter Overview

We now know how matrices arise and how to accept approximate solutions. Elimination provides the first solution route: replace the original system with two triangular systems. Pivoting addresses numerical stability; separating factorization from substitution explains reuse for multiple right-hand sides. Sparse and incomplete factorizations retain this mechanism while changing storage and approximation.

Learning Objectives Compute pivoted LU by hand; perform forward and backward substitution; distinguish factorization, solution, and pivot-growth costs.

Core Question Why can elimination be reused as a solution operator?

4.1 Elimination, Permutations, and Triangular Systems

At step k, pivot on a_{kk} and use multipliers $\ell_{ik} = a_{ik}/a_{kk}$ to update $a_{ij} \leftarrow a_{ij} - \ell_{ik}a_{kj}$. The multipliers form unit lower-triangular L, and the remaining entries form upper-triangular U. Accumulating row permutations in P gives $PA = LU$. The solution proceeds through

$$y_i = (P\boldsymbol{b})_i - \sum_{j<i} \ell_{ij} y_j, \qquad x_i = \frac{y_i - \sum_{j>i} u_{ij} x_j}{u_{ii}}.$$

Partial pivoting selects the largest-magnitude available entry in the current column. Complete pivoting searches the remaining rows and columns, giving $PAQ = LU$; the unknown ordering must then be restored. Partial pivoting is usually effective, although large element growth remains possible in the worst case.

4.2 A Complete Four-by-Four Example

Take

$$A = \begin{bmatrix} 0 & 2 & 1 & 0 \\ 4 & 4 & 0 & 1 \\ 2 & 0 & 3 & 1 \\ 0 & 1 & 1 & 2 \end{bmatrix}, \quad \boldsymbol{b} = (3, 9, 6, 4)^T.$$

Exchange the first two rows. The first-column multiplier is $\ell_{31} = 1/2$. With second pivot 2, use $\ell_{32} = -1, \ell_{42} = 1/2$; the third-column multiplier is $\ell_{43} = 1/8$. Thus

$$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1/2 & -1 & 1 & 0 \\ 0 & 1/2 & 1/8 & 1 \end{bmatrix}, \quad U = \begin{bmatrix} 4 & 4 & 0 & 1 \\ 0 & 2 & 1 & 0 \\ 0 & 0 & 4 & 1/2 \\ 0 & 0 & 0 & 31/16 \end{bmatrix}.$$

Verify $LU = PA$ entry by entry. Forward substitution gives $\boldsymbol{y} = (9, 3, 9/2, 31/16)^T$, and backward substitution gives $\boldsymbol{x} = (1, 1, 1, 1)^T$. The permutation determinant is -1, so $\det A = -62$.

Potential Conceptual Jump

Potential Question: Why can the first pivot vanish when the matrix is invertible?
Author Supplement: Invertibility ensures a solution, not nonzero elimination pivots in every ordering. Row exchanges select suitable elimination coordinates. In floating-point arithmetic, small pivots can also amplify multipliers and roundoff.
Core Conceptual Connection: Invertible system $\longrightarrow$ suitable elimination order $\longrightarrow$ numerically viable triangular factors.

4.3 Cost, Reuse, and Verification

Dense LU costs approximately $2N^3/3$ FLOPs; two triangular solves cost approximately $2N^2$, with $O(N^2)$ in-place factor storage. For s right-hand sides, factor once and substitute s times. Block LU organizes most updates as matrix multiplication, changing cache reuse without changing the underlying elimination.

Cost and Applicability Factors support multiple right-hand sides and iterative refinement. Matrix changes generally require refactorization. Report the true residual; if needed, compute residuals more accurately and solve correction equations.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Work through every trailing submatrix, forward solve, and back substitution in the four-by-four example, then cross-check $PA = LU$ with a library factorization.
Follow-up: Replace the first nonzero pivot by a small value and compare multipliers, element growth, and backward errors with and without partial pivoting.

Further Detail and Search Worst-case pivot-growth bounds and floating-point proofs for block algorithms are omitted. See [1].

Keywords: partial pivoting; complete pivoting; growth factor; blocked LU; iterative refinement

Chapter 5

Sparse LU and Large Sparse Systems

Chapter Overview

Sparsity reduces storage of the original matrix, but elimination introduces new couplings in its factors. Elimination graphs explain fill, while ordering, symbolic analysis, and numerical factorization expose its separate costs. These intermediate structures determine memory and work estimates and motivate fill truncation in incomplete factorization.

Learning Objectives Identify Schur-complement fill; explain elimination graphs and ordering; state conditional cost estimates for regular two- and three-dimensional problems.

Core Question Why can factors exceed memory when the original matrix is sparse?

5.1 Elimination Connects Neighbors

Partition one pivot from the remaining unknowns:

$$A = \begin{bmatrix} a & \boldsymbol{r}^T \\ \boldsymbol{c} & B \end{bmatrix} \quad \Rightarrow \quad S = B - \boldsymbol{c}a^{-1}\boldsymbol{r}^T.$$

Even if $B_{ij} = 0$, $c_i r_j \neq 0$ can make S_{ij} nonzero. In a symmetric graph, eliminating a vertex connects its remaining neighbors into a clique. Eliminating the center of a four-vertex star first connects the three leaves; eliminating leaves first avoids these extra edges.

Potential Conceptual Jump

Potential Question: Why does numbering affect memory without changing the physical solution?
Author Supplement: Renumbering permutes coordinates, but elimination forms Schur complements in that order. The variables retained at each step determine which interactions become new couplings. Ordering optimizes intermediate structure and the computational path.
Core Conceptual Connection: Same coupling graph $\longrightarrow$ different elimination paths $\longrightarrow$ different factor fill.

5.2 Ordering and Two-Stage Analysis

Minimum-degree orderings favor vertices with few connections. Nested dissection uses small separators to divide the graph, eliminating interiors before separators. Symbolic analysis predicts

nonzeros, elimination trees, and storage; numerical factorization computes values. Dynamic pivoting in nonsymmetric LU may alter predictions, linking actual memory to stability choices. For regular, locally coupled elliptic problems with N unknowns, suitable nested dissection, and no additional pivot-growth effects, typical estimates are

	Factorization work	Factor storage
$2D$	$O(N^{3/2})$	$O(N \log N)$
$3D$	$O(N^2)$	$O(N^{4/3})$

These estimates assume regular meshes and the stated ordering model. With n points per direction, $N \sim n^2$ in two dimensions and $N \sim n^3$ in three. A common definition of the unknown count is therefore essential to cost comparisons.

5.3 A Poisson Experiment

Generate five-point Poisson in CSR and convert to the solver's CSC format. Compare natural numbering with an approximate minimum-degree-type ordering, recording $\mathrm{nnz}(A)$, $\mathrm{nnz}(L) + \mathrm{nnz}(U)$, factorization time, and solve time. Begin the three-dimensional seven-point experiment at a small size and expand while monitoring factor storage.

Cost and Applicability Sparse direct methods often suit moderate-sized, difficult-to-precondition systems and multiple right-hand sides. Fill and communication may limit large three-dimensional problems. Record numerical refactorization, symbolic reuse, and right-hand-side amortization separately.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Eliminate a four-vertex star SPD matrix center-first and leaf-first. Write every Schur complement and draw the fill edges.
Follow-up: Compare natural ordering and COLAMD factor counts for two-dimensional Poisson. Explain why this experiment illustrates ordering rather than proving the asymptotic bounds.

Further Detail and Search Separator theorems, supernodes, and multifrontal implementation are omitted. See [1, 3].

Keywords: fill-in; elimination graph; nested dissection; symbolic factorization; multifrontal

Chapter 6

Cholesky and LDL^{T} Factorizations

Chapter Overview

Exact factorization cost depends on structure. Symmetry and positive definiteness further reduce work and storage, motivating Cholesky; symmetric indefinite problems require LDL^{T} with block pivots. This distinction supplies the conditions later needed for CG, MINRES, and incomplete Cholesky.

Learning Objectives Derive Cholesky recurrences; recognize block pivots in indefinite systems; distinguish unpivoted and permuted factorizations.

Core Question Which computations does symmetry reuse, and what does positive definiteness additionally guarantee?

6.1 Cholesky Recurrences and an Example

For a real SPD matrix, a lower-triangular L with positive diagonal satisfies $A = LL^T$. Entrywise comparison gives

$$\ell_{jj} = \sqrt{a_{jj} - \sum_{k<j} \ell_{jk}^2}, \qquad \ell_{ij} = \frac{a_{ij} - \sum_{k<j} \ell_{ik}\ell_{jk}}{\ell_{jj}}.$$

Every Schur complement remains SPD, making the square-root argument positive. For $A = \left[\begin{smallmatrix} 4 & 2 \\ 2 & 3 \end{smallmatrix}\right]$, we obtain $L = \begin{bmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{bmatrix}$. Solve $L\boldsymbol{y} = \boldsymbol{b}$, then $L^T\boldsymbol{x} = \boldsymbol{y}$. Dense factorization costs approximately $N^3/3$ FLOPs and stores one triangle. Sparse problems still require ordering and fill analysis.

6.2 Indefinite Problems and Block Pivots

A general real symmetric indefinite factorization uses

$$PAP^T = LDL^T,$$

where L is unit lower triangular and D contains 1×1 and 2×2 blocks. The matrix $\left[\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right]$ is invertible but has no nonzero first scalar pivot for unpermuted scalar LDL^{T}. Taking the whole matrix as a two-by-two block resolves this example. Stable implementations require symmetric pivoting.

Potential Conceptual Jump

Potential Question: Why is replacing Cholesky square roots by signed numbers insufficient?
Author Supplement: Indefiniteness permits both negative pivots and nonzero coupling with zero diagonals. Scalar pivots may vanish or be too small, requiring simultaneous row/column permutations and two-by-two blocks. Constraint variables in saddle-point systems are a common source.
Core Conceptual Connection: Positive-energy elimination $\longrightarrow$ indefinite coupling $\longrightarrow$ symmetric block pivots.

6.3 Physical Nullspaces and Solver Strategy

With rigid-body modes, constants, or gauge freedom, constrain a semidefinite problem or solve on a compatible subspace. Adding a small diagonal changes the problem and requires an explicit interpretation. Block elimination of saddle-point systems also produces Schur complements, used for preconditioning in Chapters 18 and 21.

Cost and Applicability Cholesky applies to SPD systems. Symmetric indefinite factorization uses pivoting for numerical stability, potentially increasing fill. Structural checks therefore examine the actual discrete fluxes, boundary treatment, and preconditioner action.

Ask AI

Ask AI: Factor the two-by-two SPD example and solve for $(1,1)^T$. Identify the failed scalar LDL^{T} step in the invertible zero-diagonal example.
Follow-up: Construct Neumann Poisson with a constant nullspace, check compatibility, and compare fixing one unknown with imposing zero mean.

Further Detail and Search Bunch–Kaufman pivot selection and sparse-supernode details are omitted. See [1].

Keywords: Cholesky; symmetric indefinite; block LDLT; saddle point; nullspace

Chapter 7

QR and Least-Squares Problems

Chapter Overview

Observed data, overdetermined constraints, and projection problems may replace exactly solvable equations with residual minimization. This chapter uses orthogonal factorization for least squares and explains accuracy loss in normal equations. QR reappears in Arnoldi and GMRES, where projection reduces a large system to a small least-squares problem.

Learning Objectives Formulate least-squares projection; compare normal equations and QR; recognize rank deficiency and orthogonalization errors.

Core Question How should an approximation be chosen when its residual cannot vanish?

7.1 Projection and Conditioning

For $A \in \mathbb{R}^{m\times n}$, $m \geq n$, minimize $\min_{\boldsymbol{x}} \|A\boldsymbol{x} - \boldsymbol{b}\|_2$. With full column rank, the normal equations $A^TA\boldsymbol{x} = A^T\boldsymbol{b}$ have a unique solution, but

$$\kappa_2(A^TA) = \kappa_2(A)^2.$$

Thin QR gives $A = Q_1R$, $Q_1^TQ_1 = I$, hence

$$R\boldsymbol{x} = Q_1^T\boldsymbol{b}, \qquad \boldsymbol{r} = (I - Q_1Q_1^T)\boldsymbol{b}.$$

Orthogonal transformations preserve the two-norm without explicitly forming A^TA.

7.2 Three Orthogonalization Mechanisms

MGS sequentially removes projections onto existing directions. It resists some cancellation effects better than classical one-pass Gram–Schmidt, although ill-conditioned bases may still require reorthogonalization. Householder reflections $H = I - 2\boldsymbol{v}\boldsymbol{v}^T/(\boldsymbol{v}^T\boldsymbol{v})$ eliminate the entries below a column pivot and support stable dense QR. Givens rotations alter two components, suiting incremental Hessenberg reduction.

Potential Conceptual Jump

Potential Question: Why not solve the smaller symmetric normal equations directly?
Author Supplement: The reduction squares singular values. Forming the product can lose information when small singular values approach roundoff scales. QR uses norm-preserving transformations at the original scale; rank deficiency also requires rank detection and a solution-selection criterion.
Core Conceptual Connection: Residual minimization $\longrightarrow$ orthogonal projection $\longrightarrow$ stable small triangular problem.

7.3 A Minimal Example and Rank Deficiency

Fit $c_0 + c_1 t$ to $(t_i, b_i) = (0,1), (1,2), (2,2)$:

$$A = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, \quad \boldsymbol{c} = (7/6, 1/2)^T, \quad \boldsymbol{r} = \boldsymbol{b} - A\boldsymbol{c} = (-1/6, 1/3, -1/6)^T.$$

Verify $A^T \boldsymbol{r} = 0$. Identical columns make the least-squares solution nonunique. Column-pivoted QR supports numerical-rank assessment, and SVD supplies a minimum-norm solution at a specified threshold. A small diagonal entry in unpivoted R is a diagnostic clue; reliable rank assessment also examines column dependence.

Cost and Applicability Dense thin QR costs approximately $2mn^2 - 2n^3/3$ FLOPs. Store Householder vectors instead of a full Q. Sparse QR can also fill in. Normal equations have a defined role when conditioning is moderate and performance is important.

Ask AI

Ask AI: Construct thin QR explicitly for the three-point fit. Verify the orthogonal residual and compare with normal equations.
Follow-up: Make the second column equal to the first plus 10^{-k} times an independent vector. Compare residuals and coefficient errors for QR, normal equations, and SVD.

Further Detail and Search Backward-stability proofs, rank-revealing QR, and SVD algorithms are omitted. See [1, 2].

Keywords: Householder QR; modified Gram Schmidt; Givens; rank revealing QR; least squares

Chapter 8

Incomplete Factorization

Chapter Overview

Fill in exact sparse factorization motivates a cheaper approximate inverse action. Incomplete factorizations select and drop entries while retaining pivot and triangular-solve constraints. This prepares preconditioner design in Part IV: an approximation is assessed through its improvement of outer convergence together with setup and application costs.

Learning Objectives Distinguish ILU0, ILUk, ILUT, and IC; explain dropping and fill; apply factors through triangular solves.

Core Question Which elimination information should limited memory retain to improve iteration?

8.1 Truncated Elimination

Exact LU can create entries at initially zero locations. ILU(0) retains the prescribed original pattern. ILU(k) retains entries by symbolic fill level: original entries have level zero; a path through pivot j proposes the sum of its levels plus one, retaining the minimum proposed level. ILUT uses numerical thresholds and row-size limits. Check each library's scaling, ordering, and pivoting definitions. With $A \approx LU = M$, the preconditioning action is

$$\boldsymbol{z} = M^{-1}\boldsymbol{r}: \qquad L\boldsymbol{y} = \boldsymbol{r}, \quad U\boldsymbol{z} = \boldsymbol{y}.$$

The explicit inverse is generally not formed.

8.2 Minimal Examples

Unpivoted tridiagonal Poisson LU has no out-of-band fill, so ILU(0) can equal exact LU. A four-vertex cycle or two-dimensional five-point matrix generates additional fill; ILU(0) discards some of it. Comparing these examples exposes the dropping mechanism. IC approximates $A \approx LL^T$ in a symmetric pattern for SPD matrices A. Dropping changes the Schur complements encountered during elimination, so IC pivot positivity must also be checked. Nonpositive pivots motivate reordering, shifts, or modified strategies.

Potential Conceptual Jump

Potential Question: Why compute a factorization that is not exact?
Author Supplement: An outer iteration needs an inexpensive transformation of correction directions, not a preconditioner that solves independently. Retaining dominant local coupling can reduce outer work, at the cost of dependency-limited triangular solves on each application.
Core Conceptual Connection: Exact elimination cost $\longrightarrow$ controlled truncation $\longrightarrow$ approximate inverse within outer iteration.

Cost and Applicability Smaller drop tolerances usually increase fill, setup, and triangular-solve costs, without guaranteeing monotonically better total time. General ILU is usually nonsymmetric and does not automatically supply the SPD preconditioner required by standard PCG. Zero pivots, small pivots, and parallel dependencies are distinct risks.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Apply ILU(0) to tridiagonal and two-dimensional five-point Poisson. List dropped entries and compare true preconditioned-solve residuals.
Follow-up: Sweep ILUT drop tolerances, recording setup, factor nonzeros, GMRES counts, and total time to identify a cost tradeoff.

Further Detail and Search Modified incomplete factorizations and spectral-equivalence proofs are omitted. See [3, 4].

Keywords: ILU(0); level of fill; ILUT; incomplete Cholesky; drop tolerance

Chapter 9

LU-SGS and Approximate Factorization

Chapter Overview

Incomplete factorization controls cost by dropping entries; LU-SGS constructs approximate factors through forward and backward sweeps. Its connection with symmetric Gauss–Seidel clarifies three roles: stationary iteration, preconditioning, and pseudo-time updating. The outer mechanism determines the task of a sweep and the convergence conditions to be checked.

Learning Objectives Expand the SGS factors; implement both sweeps; distinguish CFD diagonal approximations from linear-algebraic assumptions.

Core Question How can two local sweeps approximate global implicit coupling?

9.1 Approximate Factors and Two Triangular Solves

Write $A = D + L + U$, with L, U retaining their entries' signs. For invertible D, define

$$M = (D + L)D^{-1}(D + U) = A + LD^{-1}U.$$

Solving $M\delta\boldsymbol{x} = \boldsymbol{r}$ is equivalent to

$$(D + L)\boldsymbol{y} = \boldsymbol{r}, \qquad (D + U)\delta\boldsymbol{x} = D\boldsymbol{y}.$$

No product M need be formed. If A is symmetric and D positive definite, $U = L^T$ and M is SPD. Standard PCG also requires a fixed symmetric overall preconditioner action.

9.2 Iteration, Preconditioning, and Pseudo-Time

The stationary iteration $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + M^{-1}(\boldsymbol{b} - A\boldsymbol{x}_k)$ has error matrix $I - M^{-1}A$ and converges when its spectral radius is below 1. As a preconditioner, the same inverse action is combined by an outer Krylov method whose projection selects the correction coefficients. CFD LU-SGS often approximates diagonal blocks and Jacobian splittings of $\Delta t^{-1}M_t + J$. Pseudo-time drives the steady residual toward zero; its step is not physical-time resolution. Specify the actual diagonal terms and boundary treatment.

Potential Conceptual Jump

Potential Question: Why does LU-SGS generally differ from exact LU?
Author Supplement: Convenient triangular parts are selected first and multiplied to approximate the operator. The extra cross term reveals the approximation. Exact LU computes elimination factors whose product equals the permuted matrix.
Core Conceptual Connection: Implicit coupling $\longrightarrow$ sweep-based approximation $\longrightarrow$ outer correction of remaining error.

9.3 A Three-Point Check

For three-by-three $A = \operatorname{tridiag}(-1, 2, -1)$, calculate $LD^{-1}U$ and compare $M^{-1}\boldsymbol{r}$ with $A^{-1}\boldsymbol{r}$. Sequential sweeps use updated neighboring values. Multicolor or block-Jacobi parallelization changes the operator and requires renewed evaluation.

Cost and Applicability Each application uses approximately two sparse triangular solves with $O(\operatorname{nnz} A)$ storage. Sequential dependence limits throughput. Local blocks or subdomains increase parallelism but generally weaken treatment of global coupling.

Ask AI

Ask AI: For three-point Poisson, compute M, $M^{-1}\boldsymbol{r}$, and the error iteration matrix. Distinguish one preconditioner application from stationary convergence.
Follow-up: Replace the sweep with red–black or block updates, identify the changed splitting, and compare outer GMRES counts.

Connections Chapter 17 unifies stationary iterations; Chapter 18 addresses parallel subdomains.

Further Detail and Search Compressible-flow Jacobian splittings and pseudo-time parameter estimates are omitted. See [3].

Keywords: LU-SGS; symmetric Gauss Seidel; approximate factorization; pseudo time stepping

Part III

Krylov Subspace Methods

Chapter 10

The Krylov Subspace Idea

Chapter Overview

Exact and approximate factorizations require factor access; large problems may make matrix–vector products easier. This chapter constructs Krylov spaces from residual corrections and turns solution into a growing projection problem. Arnoldi, GMRES, and CG share this space but use different bases and projection conditions. Operator applications express progressively richer corrections.

Learning Objectives Construct a subspace from the residual; derive residual polynomials; distinguish Galerkin and minimum-residual conditions.

Core Question Given only $A\boldsymbol{v}$, how can we approximate $A^{-1}\boldsymbol{b}$?

10.1 Generating a Search Space

Starting with $\boldsymbol{r}_0 = \boldsymbol{b} - A\boldsymbol{x}_0$, seek

$$\boldsymbol{x}_m \in \boldsymbol{x}_0 + \mathcal{K}_m(A, \boldsymbol{r}_0), \qquad \mathcal{K}_m = \operatorname{span}\{\boldsymbol{r}_0, A\boldsymbol{r}_0, \ldots, A^{m-1}\boldsymbol{r}_0\}.$$

If $\boldsymbol{x}_m - \boldsymbol{x}_0 = q_{m-1}(A)\boldsymbol{r}_0$, then

$$\boldsymbol{r}_m = p_m(A)\boldsymbol{r}_0, \quad p_m(z) = 1 - zq_{m-1}(z), \quad p_m(0) = 1.$$

The method selects a polynomial that reduces the relevant error or residual; no explicit high powers are required.

Potential Conceptual Jump

Potential Question: How do repeated applications of A approximate an inverse action?
Author Supplement: The space retains directions such as $A^k\boldsymbol{r}_0$ and projection selects their coefficients jointly. In a finite-dimensional invertible system, the minimal polynomial relative to the initial residual has a nonzero constant term, allowing a finite polynomial representation of the inverse action. Finite precision, restarting, and preconditioning determine the practical approximation process.
Core Conceptual Connection: Accessible operator action $\longrightarrow$ polynomial correction space $\longrightarrow$ projected coefficients.

10.2 Projection and Minimal Polynomials

With trial basis V_m and test basis W_m, impose

$$W_m^T(\boldsymbol{r}_0 - AV_m\boldsymbol{y}) = 0, \qquad W_m^T AV_m\boldsymbol{y} = W_m^T\boldsymbol{r}_0.$$

Galerkin uses the search space as its test space. Minimizing the residual two-norm requires orthogonality to $A\mathcal{K}_m$. These conditions differ. If the least degree ν satisfies $\mu_\nu(A)\boldsymbol{r}_0 = 0$, suitable unrestarted methods terminate in at most $\nu \leq N$ exact-arithmetic steps. Practical usefulness requires far fewer than N steps.

10.3 A Two-by-Two Calculation

Take $A = \mathrm{diag}(1, 2)$, $\boldsymbol{b} = (1, 1)^T$, and $\boldsymbol{x}_0 = 0$. The first space contains only $\alpha\boldsymbol{b}$, with minimum-residual coefficient $\alpha = 3/5$. The second, spanned by $(1, 1)^T, (1, 2)^T$, is all of $\mathbb{R}^2$ and contains $(1, 1/2)^T$. For $\boldsymbol{b} = (1, 0)^T$, one dimension suffices: the initial residual affects effective difficulty.

Cost and Applicability Each step requires at least one operator application. Basis storage and orthogonalization depend on the method. Matrix-free and distributed operators fit directly; preconditioning often matters more to feasibility than fine details of the original spectrum.

Ask AI

Ask AI: Compute the first Galerkin and minimum-residual steps for the two-by-two example and explain their different coefficients.
Follow-up: Use $\mu(z) = (z - 1)(z - 2)$ to express the inverse as a polynomial. Explain degree reduction when the residual has one eigenvector component.

Further Detail and Search General projection stability bounds and nonnormal polynomial approximation are omitted. See [3, 2].

Keywords: Krylov subspace; residual polynomial; minimal polynomial; Galerkin projection

Chapter 11

Arnoldi and Hessenberg Decomposition

Chapter Overview

Krylov powers generate a search space, but increasing linear dependence complicates numerical use. Arnoldi orthogonalizes each new direction and represents the large operator in a small Hessenberg matrix. This verifiable connection between large and small systems supplies the least-squares problem used by GMRES in the next chapter.

Learning Objectives Perform MGS Arnoldi; calculate two steps; explain loss of orthogonality and happy breakdown.

Core Question How can a reliable subspace basis be constructed without storing high matrix powers?

11.1 The Arnoldi Recurrence

Set $\beta = \|\boldsymbol{r}_0\|_2$, $\boldsymbol{v}_1 = \boldsymbol{r}_0/\beta$. At step j,

$$\boldsymbol{w} = A\boldsymbol{v}_j; \quad h_{ij} = \boldsymbol{v}_i^T \boldsymbol{w}, \quad \boldsymbol{w} \leftarrow \boldsymbol{w} - h_{ij}\boldsymbol{v}_i \quad (i = 1, \ldots, j); \qquad h_{j+1,j} = \|\boldsymbol{w}\|_2 .$$

If $h_{j+1,j} \neq 0$, set $\boldsymbol{v}_{j+1} = \boldsymbol{w}/h_{j+1,j}$. Consequently,

$$AV_m = V_{m+1}\bar{H}_m, \qquad V_{m+1}^T V_{m+1} = I$$

in exact arithmetic, where $\bar{H}_m$ is $(m+1) \times m$ and upper Hessenberg.

11.2 Two Steps for the Four-by-Four Matrix

Use A from Chapter 4 and $\boldsymbol{r}_0 = \boldsymbol{e}_1$. First,

$$A\boldsymbol{e}_1 = (0, 4, 2, 0)^T, \quad h_{11} = 0, \quad h_{21} = 2\sqrt{5}, \quad \boldsymbol{v}_2 = (0, 2, 1, 0)^T/\sqrt{5}.$$

Next, $A\boldsymbol{v}_2 = (5, 8, 3, 3)^T/\sqrt{5}$. Sequential projection gives

$$h_{12} = \sqrt{5}, \quad h_{22} = 19/5, \quad h_{32} = 7/5, \quad \boldsymbol{v}_3 = (0, 2, -4, 15)^T/(7\sqrt{5}),$$

$$\bar{H}_2 = \begin{bmatrix} 0 & \sqrt{5} \\ 2\sqrt{5} & 19/5 \\ 0 & 7/5 \end{bmatrix}.$$

Check $\boldsymbol{v}_i^T \boldsymbol{v}_j = \delta_{ij}$ and $AV_2 = V_3 \bar{H}_2$.

Potential Conceptual Jump

Potential Question: How does a small matrix retain solution information from the original problem?
Author Supplement: Arnoldi represents the action on generated basis vectors exactly in the extended basis. The later approximation comes from restricting the trial space, not from approximating these represented actions. Finite precision also requires monitoring orthogonality.
Core Conceptual Connection: Large operator action $\longrightarrow$ orthogonal coordinates $\longrightarrow$ small Hessenberg problem.

11.3 Breakdown Diagnostics and Cost

The condition $h_{j+1,j} = 0$ identifies an A-invariant subspace. For an invertible system and the current residual, residual minimization then gives the exact solution: happy breakdown. In finite precision, a nearly zero subdiagonal instead triggers a true-residual check against the convergence tolerance.

Cost and Applicability m steps require approximately m matrix products, $O(Nm^2)$ orthogonalization work, and $O(Nm)$ storage. MGS can lose orthogonality; repeat projection or use a more stable strategy when needed. Distributed inner products incur reductions.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Recompute both steps in MGS order, compare AV_2 with $V_3\bar{H}_2$, and verify every inner product.
Follow-up: Perturb a power basis by roundoff and compare orthogonality after classical Gram–Schmidt, MGS, and two-pass MGS.

Further Detail and Search Selective reorthogonalization and communication-avoiding Arnoldi are omitted. See [3].

Keywords: Arnoldi; Hessenberg; reorthogonalization; happy breakdown; orthogonality loss

Chapter 12

GMRES

Chapter Overview

Arnoldi supplies an orthogonal basis and a small Hessenberg representation. GMRES chooses the correction that minimizes the Euclidean residual in this space. Incremental least squares leads to the algorithm, and restarting balances storage against convergence. Projected residual estimates monitor iteration, while true residuals verify the original equation.

Learning Objectives Derive GMRES least squares; explain Givens rotations and restart; interpret preconditioned residuals correctly.

Core Question How is the best linear combination chosen from an orthogonal basis?

12.1 From a Large Residual to Small Least Squares

Write $\boldsymbol{x}_m = \boldsymbol{x}_0 + V_m \boldsymbol{y}$. Then

$$\boldsymbol{r}_m = V_{m+1}(\beta \boldsymbol{e}_1 - \bar{H}_m \boldsymbol{y}), \qquad \boldsymbol{y}_m = \arg\min_{\boldsymbol{y}} \left\| \beta \boldsymbol{e}_1 - \bar{H}_m \boldsymbol{y} \right\|_2 .$$

Norm equality depends on orthogonality of V_{m+1}. On each new column, Givens rotations eliminate the subdiagonal entry and rotate the right-hand side, updating the residual estimate. Back substitution finally gives $\boldsymbol{y}_m$; the full solution need not be formed at every inner iteration. For Chapter 11's two-step example with $\beta = 1$, the least-squares solution is

$$y_1 = -95/348, \qquad y_2 = 25\sqrt{5}/174, \qquad \|\boldsymbol{r}_2\|_2 = 7/\sqrt{174}.$$

This completes a directly checkable Arnoldi–least-squares chain.

12.2 Restart and Preconditioning Semantics

GMRES(m) updates the solution after m steps and restarts from its new residual. Retaining only m directions limits storage but discards prior subspace information and can cause stagnation. Larger restart lengths increase orthogonalization and global communication. Left preconditioning solves $M^{-1}A\boldsymbol{x} = M^{-1}\boldsymbol{b}$, usually minimizing $\left\|M^{-1}\boldsymbol{r}\right\|$. Right preconditioning retains the true-residual objective. Verify stopping and callback semantics separately for each library.

Potential Conceptual Jump

Potential Question: Why recompute a residual after its monitored curve has decreased?
Author Supplement: The least-squares estimate relies on the Arnoldi relation and preconditioner scaling. Roundoff perturbs the relation, while left preconditioning changes the norm being monitored. The original equation's residual is the quantity linked to error in Chapter 3.
Core Conceptual Connection: Projected-residual monitoring $\longrightarrow$ library semantics and scaling $\longrightarrow$ acceptance of the original equation.

12.3 The Computational Interface

The SciPy experiment uses left preconditioning, with `M` representing the inverse preconditioner action. At each inner iteration, `callback_type='pr_norm'` reports a preconditioned residual quantity; final verification evaluates $\boldsymbol{b} - A\boldsymbol{x}$. Record restart length, maximum cycles, and tolerances, distinguishing cycles from inner iterations.[21]

Cost and Applicability A cycle uses roughly m operator and preconditioner applications, $O(Nm^2)$ orthogonalization, and $O(Nm)$ basis storage. For nonsymmetric systems, tune restart and preconditioner quality together.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Apply two Givens rotations explicitly to this chapter's 3×2 least-squares problem. Verify its coefficients and residual norm.
Follow-up: Compare GMRES(10), GMRES(30), and ILU-GMRES in total time and true residual. Report inner iterations and restart cycles separately.

Further Detail and Search Stagnation constructions, harmonic Ritz vectors, and recycled subspaces are omitted. See [3, 4].

Keywords: restarted GMRES; Givens QR; preconditioned residual; deflated GMRES

Chapter 13

Conjugate Gradients

Chapter Overview

General nonsymmetric problems require long bases or restart, whereas SPD structure supports conjugate directions with short recurrences. This chapter develops CG through quadratic-energy minimization, states its assumptions and error bound, and completes a two-step example. Later SPD preconditioners and multigrid combinations must preserve the required structure.

Learning Objectives Execute CG and PCG; explain A-conjugacy and energy error; identify standard PCG assumptions.

Core Question How does SPD structure reduce storage while retaining subspace optimality?

13.1 Energy Minimization and Short Recurrences

For SPD A, CG minimizes $\|\boldsymbol{x}_* - \boldsymbol{x}\|_A$ over $\boldsymbol{x}_0 + \mathcal{K}_k(A, \boldsymbol{r}_0)$, where $\|\boldsymbol{e}\|_A^2 = \boldsymbol{e}^T A \boldsymbol{e}$. Initialize $\boldsymbol{p}_0 = \boldsymbol{r}_0$ and iterate

$$\alpha_k = \frac{\boldsymbol{r}_k^T \boldsymbol{r}_k}{\boldsymbol{p}_k^T A \boldsymbol{p}_k}, \quad \boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \alpha_k \boldsymbol{p}_k, \quad \boldsymbol{r}_{k+1} = \boldsymbol{r}_k - \alpha_k A \boldsymbol{p}_k,$$

$$\beta_k = \frac{\boldsymbol{r}_{k+1}^T \boldsymbol{r}_{k+1}}{\boldsymbol{r}_k^T \boldsymbol{r}_k}, \qquad \boldsymbol{p}_{k+1} = \boldsymbol{r}_{k+1} + \beta_k \boldsymbol{p}_k.$$

In exact arithmetic, residuals are mutually orthogonal and directions are A-conjugate. Positive definiteness makes each nonzero search-direction denominator positive, supporting the short recurrence.

13.2 Two Steps and an Error Bound

Take $A = \left[\begin{smallmatrix} 4 & 1 \\ 1 & 3 \end{smallmatrix}\right]$, $\boldsymbol{b} = (1, 2)^T$, $\boldsymbol{x}_0 = 0$. The first step has $\alpha_0 = 1/4$, $\boldsymbol{x}_1 = (1/4, 1/2)^T$, $\boldsymbol{r}_1 = (-1/2, 1/4)^T$; $\beta_0 = 1/16$, $\boldsymbol{p}_1 = (-7/16, 3/8)^T$. The second has $\alpha_1 = 4/11$, giving $\boldsymbol{x}_2 = (1/11, 7/11)^T$. A classical bound is

$$\frac{\|\boldsymbol{e}_k\|_A}{\|\boldsymbol{e}_0\|_A} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k, \qquad \kappa = \kappa_2(A).$$

This is a worst-case upper bound. Eigenvalue clustering and right-hand-side directions may yield faster convergence; the residual two-norm need not decrease monotonically.

13.3 Conditions for PCG

For fixed SPD M, solve $\boldsymbol{z}_k = M^{-1}\boldsymbol{r}_k$. Replace the numerator of α by $\boldsymbol{r}_k^T\boldsymbol{z}_k$ and define β by ratios of successive such inner products, using direction $\boldsymbol{z}_{k+1} + \beta_k\boldsymbol{p}_k$. The effective SPD operator is $M^{-1/2}AM^{-1/2}$; Euclidean symmetry of $M^{-1}A$ is not required.

Potential Conceptual Jump

Potential Question: Why does SPD of the original matrix not justify arbitrary preconditioning in standard CG?
Author Supplement: Preconditioning changes the geometry used by the method. A fixed SPD inverse action defines an admissible energy-coordinate transformation. Nonsymmetric or iteration-dependent actions generally disrupt the orthogonality behind short recurrences.
Core Conceptual Connection: Original energy $\longrightarrow$ symmetric positive-definite transformation $\longrightarrow$ preserved conjugacy structure.

Cost and Applicability Each step uses one $A\boldsymbol{p}$, a few vector updates and dot products, and $O(N)$ storage; distributed reduction latency may dominate. For ILU or AMG, check the full inverse action against standard PCG's fixed, symmetric, positive-definite requirements.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: Compute every inner product in the two-step example. Verify A-conjugacy and compare energy error with the residual two-norm.
Follow-up: Apply IC and nonsymmetric ILU to the same SPD system. Check assumptions explicitly; one successful run does not establish applicability.

Further Detail and Search Finite-precision conjugacy loss and pipelined-CG stability proofs are omitted. See [3, 4].

Keywords: conjugate gradient; A norm; PCG; SPD preconditioner; finite precision CG

Chapter 14

BiCG and BiCGSTAB

Chapter Overview

GMRES basis storage and orthogonalization can be expensive for nonsymmetric systems. This chapter introduces BiCG through two-sided spaces and BiCGSTAB through local residual smoothing. Short recurrences reduce storage but introduce nonmonotone convergence and breakdown risks. Selection must account for memory, matrix structure, and failure handling.

Learning Objectives Interpret shadow residuals and biorthogonality; execute BiCGSTAB; distinguish breakdown from convergence.

Core Question Can nonsymmetric systems retain short recurrences, and at what cost?

14.1 Two-Sided Projection

BiCG builds $\mathcal{K}_k(A, \boldsymbol{r}_0)$ and $\mathcal{K}_k(A^T, \widetilde{\boldsymbol{r}}_0)$, with biorthogonal residuals and biconjugate directions. The shadow residual must satisfy $\widetilde{\boldsymbol{r}}_0^T \boldsymbol{r}_0 \neq 0$. The method usually needs both A and A^T, and crucial inner products can vanish before the true residual does. BiCGSTAB multiplies the BiCG residual polynomial by local linear factors, avoiding explicit A^T action and reducing some oscillations. It does not guarantee residual decrease at every step.

14.2 An Executable Recurrence

For a fixed shadow $\widetilde{\boldsymbol{r}}$, initialize $\boldsymbol{p} = \boldsymbol{v} = 0$, $\rho_{\text{old}} = \alpha = \omega = 1$. At each step compute

$$\rho = \widetilde{\boldsymbol{r}}^T \boldsymbol{r}, \quad \beta = (\rho/\rho_{\text{old}})(\alpha/\omega), \quad \boldsymbol{p} = \boldsymbol{r} + \beta(\boldsymbol{p} - \omega \boldsymbol{v}), \quad \boldsymbol{v} = A\boldsymbol{p}, \quad \alpha = \rho/(\widetilde{\boldsymbol{r}}^T \boldsymbol{v}),$$

$$\boldsymbol{s} = \boldsymbol{r} - \alpha \boldsymbol{v}, \qquad \boldsymbol{t} = A\boldsymbol{s}, \qquad \omega = \frac{\boldsymbol{t}^T \boldsymbol{s}}{\boldsymbol{t}^T \boldsymbol{t}}, \qquad \boldsymbol{x} \leftarrow \boldsymbol{x} + \alpha \boldsymbol{p} + \omega \boldsymbol{s}, \quad \boldsymbol{r} \leftarrow \boldsymbol{s} - \omega \boldsymbol{t}.$$

If $\boldsymbol{s}$ already meets tolerance, update by $\alpha \boldsymbol{p}$ and stop before a 0/0. Check ρ, $\widetilde{\boldsymbol{r}}^T \boldsymbol{v}$, $\boldsymbol{t}^T \boldsymbol{t}$, and ω for breakdown. With right preconditioning, transform actual directions by M^{-1} and use those directions in solution updates; replacing only one matrix product is insufficient.

Potential Conceptual Jump

Potential Question: What does the short recurrence save, and what does it give up?
Author Supplement: GMRES stores its current basis and minimizes over the whole current search space. BiCGSTAB retains few vectors through special polynomial recurrences, giving up that stepwise minimum-residual property and depending on potentially degenerate inner products.
Core Conceptual Connection: Full-basis minimization $\longrightarrow$ short-recurrence selection $\longrightarrow$ storage savings and breakdown risks.

14.3 A Minimal Experiment

Use a one-dimensional upwind advection–diffusion tridiagonal matrix, prescribe $\boldsymbol{x}_* = \mathbf{1}$, and set $\boldsymbol{b} = A\boldsymbol{x}_*$. Compare GMRES and BiCGSTAB true-residual histories, operator counts, and times. Vary diffusion and right-hand sides to probe nonnormality and residual directions.

Cost and Applicability A typical step uses two matrix and preconditioner applications with $O(N)$ basis storage. Per-step cost and observed convergence determine total work. On failure, record the return code and true residual before adjusting the preconditioner, restarting the shadow vector, or switching to GMRES.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py`

Ask AI

Ask AI: For $A = \left[\begin{smallmatrix} 3 & 1 \\ 0 & 2 \end{smallmatrix}\right]$ and $\boldsymbol{b} = (1,1)^T$, calculate the first step and identify the local residual minimized by ω.
Follow-up: Construct a near-breakdown example. Use scale-aware denominator checks and distinguish early convergence from genuine failure.

Further Detail and Search Full two-sided BiCG derivation and stabilized-polynomial variants are omitted. See [3, 4].

Keywords: BiCG; biorthogonality; shadow residual; BiCGSTAB; breakdown

Chapter 15

Choosing and Extending Krylov Methods

Chapter Overview

The preceding chapters established common Krylov spaces and principal recurrences. Matrix structure, projection objectives, and preconditioner variability now organize extensions to symmetric indefinite systems, two-sided projection, flexible preconditioning, and multiple right-hand sides. These distinctions place each algorithm within an appropriate solver workflow.

Learning Objectives Match assumptions to MINRES, FOM, QMR, TFQMR, and FGMRES; explain block and recycled Krylov spaces.

Core Question Which structural differences define the methods, and when must the outer iteration change?

15.1 Structure and Projection Choices

Method	Structure and objective	Important conditions
MINRES	Real symmetric, possibly indefinite; minimum residual	Standard preconditioning must be fixed SPD, not arbitrarily indefinite
FOM	Galerkin condition on an Arnoldi basis	The projected system may be singular; residual need not be minimal
QMR	Two-sided Lanczos quasi-minimum residual	Usually needs transpose action; two-sided breakdown remains possible
TFQMR	Transpose-free quasi-minimum residual	Short recurrence and monitoring differ from GMRES
FGMRES	Variable or nonlinear preconditioning	Stores actual correction directions, increasing memory
Block Krylov	Several related right-hand sides	Block rank deficiency, orthogonalization, and storage costs

Symmetric semidefinite and singular problems additionally require compatibility and nullspace handling; invertible-case conclusions do not replace these conditions.

15.2 Actual Directions in FGMRES

At right-preconditioned step j, take $\boldsymbol{z}_j = M_j^{-1}\boldsymbol{v}_j$ and orthogonalize $A\boldsymbol{z}_j$, obtaining

$$AZ_m = V_{m+1}\bar{H}_m, \qquad \boldsymbol{x}_m = \boldsymbol{x}_0 + Z_m\boldsymbol{y}_m.$$

For fixed linear preconditioning, Z_m is the image of the basis under one inverse action. Variable-accuracy inner iterations, changing AMG, or nonlinear smoothing generally lack that interpretation. FGMRES stores actual $\boldsymbol{z}_j$, retaining a valid projection relation.[20]

Potential Conceptual Jump

Potential Question: Why use a flexible outer method if the inner problems are nearly identical?
Author Supplement: Residual-dependent inner stopping need not define the same fixed linear map on every call. An outer method assuming a fixed preconditioner can then mismatch its basis and update relations. A flexible method retains the directions actually generated.
Core Conceptual Connection: Changing inner action $\longrightarrow$ actual correction directions $\longrightarrow$ flexible projection.

15.3 Combinations and Minimal Examples

Compare MINRES and GMRES on a symmetric indefinite two-by-two system. For Poisson, alternate one and two relaxation sweeps and write the first two columns of Z_m. Block methods share information across right-hand sides; recycling can help consecutive time steps or parameter solves, with additional storage and update costs.

Cost and Applicability First verify matrix structure, then determine whether preconditioning is fixed and symmetric, and compare operator counts, reductions, and storage. Return codes, true residuals, and physical errors form the final verification record.

Ask AI

Ask AI: Write the first MINRES and CG steps for a symmetric indefinite two-by-two system. Explain why MINRES applies but CG's positive-definiteness argument fails.
Follow-up: Derive the FGMRES least-squares problem from $AZ_m = V_{m+1}\bar{H}_m$ and explain why the update uses Z_m, not V_m.

Further Detail and Search Lanczos biorthogonality proofs, block rank revelation, and recycling algorithms are omitted. See [3, 4, 20].

Keywords: MINRES; FOM; QMR; TFQMR; flexible GMRES; block Krylov; recycling

Part IV

Preconditioning and Multilevel Methods

Chapter 16

The Role and Cost of Preconditioning

Chapter Overview

Krylov methods build corrections through operator applications; preconditioning further adjusts directional scales and coupling. Left, right, and symmetric transformations connect approximate factorization to projection. Their value depends on transformed convergence, inverse-action cost, and reuse, with conditioning, spectral clustering, and nonnormality playing different roles for different matrix classes.

Learning Objectives Distinguish left, right, and symmetric preconditioning; apply inverse actions correctly; include setup and application in total cost.

Core Question How can a cheaper problem improve the original solve without changing its exact solution?

16.1 Transformations and Residuals

A fixed invertible M can be used as

$$M^{-1}A\boldsymbol{x} = M^{-1}\boldsymbol{b} \quad \text{(left)}, \qquad AM^{-1}\boldsymbol{y} = \boldsymbol{b},\ \boldsymbol{x} = M^{-1}\boldsymbol{y} \quad \text{(right)}.$$

If $M = CC^T$ is SPD, use the symmetric transformation $C^{-1}AC^{-T}\widehat{\boldsymbol{x}} = C^{-1}\boldsymbol{b}$, $\boldsymbol{x} = C^{-T}\widehat{\boldsymbol{x}}$. Apply triangular or approximate solves without forming M^{-1}. For SPD matrices, spectral equivalence $c\,\boldsymbol{v}^T M\boldsymbol{v} \le \boldsymbol{v}^T A\boldsymbol{v} \le C\,\boldsymbol{v}^T M\boldsymbol{v}$ bounds the effective condition number by C/c. In nonsymmetric systems, reduced conditioning alone does not ensure faster GMRES; eigenvectors and nonnormality also affect residual polynomials.

Potential Conceptual Jump

Potential Question: Why can left and right preconditioning have identical eigenvalues but different residual histories?
Author Supplement: For invertible M, the preconditioned matrices are similar, but variables, residual metrics, and polynomial-action vectors differ. Similarity does not preserve Euclidean norms. Specify the minimized quantity and the original equation checked at termination.
Core Conceptual Connection: Equivalent exact solution $\longrightarrow$ different projection geometries $\longrightarrow$ different residual histories.

16.2 A Minimal Example and Cost

For $A = \mathrm{diag}(1, 10^6)$ and $M = A$, preconditioning removes the scale disparity in one action. For a general large matrix, however, solving exactly with $M = A$ is the original problem. Jacobi retains the diagonal, block methods retain local coupling, and multigrid approximately handles cross-scale coupling.

$$T_{\text{total}} = T_{\text{setup}} + k(T_A + T_{M^{-1}} + T_{\text{orth}} + T_{\text{reduce}}) + T_{\text{verify}}.$$

Multiple right-hand sides or time steps can amortize setup; coefficient changes require reassessing reuse.

Cost and Applicability Control storage, setup, application, communication, and robustness together. Fewer iterations with expensive triangular solves can cost more than additional inexpensive iterations.

Ask AI

Ask AI: Implement left- and right-preconditioned GMRES for a nonsymmetric three-by-three system. Plot true and preconditioned residuals and explain the difference.
Follow-up: Given setup and per-step costs for two preconditioners, find the number of right-hand sides at which the more expensive setup pays off.

Further Detail and Search Field-of-values bounds and general spectral-equivalence proofs are omitted. See [3, 4].

Keywords: left preconditioning; right preconditioning; spectral equivalence; time to solution

Chapter 17

Stationary Iteration and Smoothing

Chapter Overview

Preconditioners can arise from simple iterations as well as factorizations. This chapter expresses Jacobi, Gauss–Seidel, and SOR as error-propagation operators, explaining why a slow standalone solver can be an effective smoother. Multigrid relies on assigning local iteration only the error components it removes well.

Learning Objectives Form iteration matrices; use spectral radii for convergence; distinguish high-frequency smoothing from a complete solve.

Core Question Why can a slow solver be an effective algorithmic component?

17.1 Matrix Splittings

For $A = M - N$, iterate $\boldsymbol{x}_{k+1} = M^{-1}(N\boldsymbol{x}_k + \boldsymbol{b})$. The error matrix is $E = M^{-1}N = I - M^{-1}A$. Convergence for every initial error is equivalent to $\rho(E) < 1$. With $A = D + L + U$:

Jacobi	$M = D$
Weighted Jacobi	$\boldsymbol{x}^+ = \boldsymbol{x} + \omega D^{-1}\boldsymbol{r}$
Gauss–Seidel	$M = D + L$
SOR	$M = D/\omega + L$

Gauss–Seidel converges for SPD matrices; SOR does so for SPD matrices with $0 < \omega < 2$. General matrices require additional conditions; sparsity alone is insufficient.

17.2 A Fourier Smoothing Example

For interior one-dimensional Poisson, analyze local periodic modes $e_i = e^{\mathrm{i}i\theta}$, ignoring physical-boundary effects on the local symbol. Weighted Jacobi has error amplification

$$\widehat{E}(\theta) = 1 - \omega(1 - \cos\theta).$$

With $\omega = 2/3$, $|\widehat{E}| \le 1/3$ for $\theta \in [\pi/2, \pi]$, while $\widehat{E} \to 1$ as $\theta \to 0$. Oscillatory error decays rapidly, but smooth error remains. Ordinary Jacobi can merely reverse the highest frequency's sign without damping it; relaxation is not automatically effective smoothing.

Potential Conceptual Jump

Potential Question: Why do high and low frequencies need different treatments?
Author Supplement: Local updates compare nearby values. Oscillatory errors produce large residuals and are readily corrected; slowly varying errors have small fine-grid residuals and need long-range communication. Coarsening turns slow variation into a more tractable scale.
Core Conceptual Connection: Local relaxation $\longrightarrow$ remaining smooth error $\longrightarrow$ coarse-space correction.

17.3 Implementation and Cost

Start with $e_i = \sin(\pi x_i) + 0.3\sin(16\pi x_i)$ and apply weighted Jacobi to the homogeneous error equation. Project onto each component separately. Sequential GS reads updated values; Jacobi reads an old vector. Preserve this distinction in parallel implementations.

Cost and Applicability Jacobi is easily parallelized and usually bandwidth-sensitive; GS has dependencies. Multicolor, block, and Chebyshev smoothers change parallelism and frequency response, requiring operator-specific parameters.

Ask AI

Ask AI: Derive $\widehat{E}(\theta)$ from three-point Poisson. Sweep ω and compare low- and high-frequency damping.
Follow-up: Introduce anisotropic diffusion, explain point-smoothing failure, and propose line relaxation or semicoarsening.

Further Detail and Search The complete two-grid local Fourier symbol is omitted. See [5, 3].
Keywords: weighted Jacobi; smoothing factor; local Fourier analysis; line relaxation

Chapter 18

Incomplete Factorization and Domain Decomposition

Chapter Overview

Local triangular solves and sweeps do not directly scale to many processors. Partitioning unknowns into subdomains combines local solves and overlapping exchange into parallel preconditioners. Once local coupling is treated, global low-frequency errors still require a coarse space, motivating multigrid.

Learning Objectives Write additive Schwarz; distinguish overlap, RAS, and symmetry; explain the need for a coarse space.

Core Question How can approximate inverse action be parallelized without losing essential global information?

18.1 Combining Local Solves

Let R_i restrict a global vector to a subdomain, with $A_i = R_i A R_i^T$. Additive Schwarz applies

$$B = \sum_i R_i^T A_i^{-1} R_i, \qquad \boldsymbol{z} = B\boldsymbol{r}.$$

Subdomains may overlap; local systems can use exact LU, ILU, or other approximations. For SPD A, covering subdomains, and exact or SPD local inverse actions, B can be SPD. Multiplicative Schwarz updates residuals sequentially, resembling subdomain GS. RAS injects local corrections only into nonoverlapping cores, reducing repeated exchange but generally producing a nonsymmetric inverse action. Outer-method selection therefore checks both the original matrix and the RAS action against standard PCG's symmetry requirements.

18.2 Coarse Spaces and Block Systems

Single-level local methods transmit smooth error slowly across many subdomains. Adding coarse space Z,

$$B_0 = Z(Z^T A Z)^{-1} Z^T,$$

addresses global modes and can be combined with local corrections. Without a coarse level, iteration counts often rise with the subdomain count; bounds depend on overlap, scales, and the PDE. For saddle-point systems, block elimination yields the Schur complement $S = A_{22} -$

$A_{21}A_{11}^{-1}A_{12}$. Approximate block-diagonal or block-triangular preconditioners can address separate physical scales instead of applying one generic ILU to all entries.

Potential Conceptual Jump

Potential Question: Why can accurate local solutions still yield slow global convergence?
Author Supplement: Local accuracy treats internal subdomain relations. Slowly varying interdomain modes may survive every individual correction. A coarse space exchanges global information; greater local accuracy alone does not replace it.
Core Conceptual Connection: Accurate local solve $\longrightarrow$ remaining interdomain modes $\longrightarrow$ global coarse correction.

18.3 A Minimal Experiment

Partition one-dimensional Poisson into four subdomains and compare block Jacobi, one-layer-overlap additive Schwarz, and a version with a subdomain-constant coarse space. At fixed total degrees of freedom, increase subdomain count and record local setup, exchange volume, and iterations to expose the role of coarse correction. Distributed scaling is then measured through multiprocess timings.

Cost and Applicability Overlap increases local storage and communication but may improve convergence. Local ILU confines sequential dependencies to subdomains; coarse solves and global reductions can become the bottleneck.

Ask AI

Ask AI: For eight-point Poisson, write two overlapping restrictions R_i and B. Check symmetry, then replace injection by RAS and check again.
Follow-up: Derive the Schur complement of a pressure–velocity system and explain why block-triangular preconditioning often accompanies GMRES.

Further Detail and Search Schwarz condition-number bounds, BDDC, and FETI-DP are omitted. See [3, 20].

Keywords: additive Schwarz; restricted additive Schwarz; domain decomposition; coarse space; Schur complement

Chapter 19

Geometric Multigrid

Chapter Overview

Smoothing removes oscillatory error, while domain decomposition reveals the difficulty of smooth global modes. Geometric multigrid combines fine-grid smoothing with coarse correction: restrict the fine residual, solve a coarse error equation, and prolong the correction. Its efficiency comes from treating each error scale in a suitable space.

Learning Objectives Derive two-grid error propagation; define R, P, and Galerkin coarse operators; distinguish V-cycles, W-cycles, and FMG.

Core Question How can each error scale be treated on a suitable grid?

19.1 Error Equations and Coarse Correction

The residual is $\boldsymbol{r}_h = \boldsymbol{b}_h - A_h\boldsymbol{x}_h = A_h\boldsymbol{e}_h$. With interpolation P and restriction R, define

$$A_H = RA_hP, \quad A_H\boldsymbol{e}_H = R\boldsymbol{r}_h, \quad \boldsymbol{x}_h \leftarrow \boldsymbol{x}_h + P\boldsymbol{e}_H.$$

Exact coarse solution has error operator $I - PA_H^{-1}RA_h$. With pre- and postsmoothing $S_{\text{pre}}, S_{\text{post}}$, the two-grid operator is

$$E_{\text{TG}} = S_{\text{post}}(I - PA_H^{-1}RA_h)S_{\text{pre}}.$$

For SPD systems, take $R = P^T$. Suitable symmetric smoothing and positive-definite coarse solution can provide a fixed SPD preconditioner; arbitrary pre/post combinations do not ensure this.

19.2 One-Dimensional Interpolation and Recursion

The fine grid contains $N = 2N_H + 1$ interior points. Interpolation injects at coincident coarse nodes and averages between them, with homogeneous error boundaries. Taking $R = P^T$ fixes the restriction normalization; any overall scaling of R must also scale the coarse operator and right-hand side. A V-cycle recurses once per level and a W-cycle twice. FMG starts with a coarse solution, prolongs through successive levels, and applies cycles to obtain an initial approximation near discretization accuracy. The first two specify recursive cycles; FMG organizes the coarse-to-fine solution process.

Potential Conceptual Jump

Potential Question: How can a lower-resolution grid improve the fine solution?
Author Supplement: The coarse grid represents error that remains slowly varying on the fine grid; high-frequency detail is assigned to smoothing. Its right-hand side is the restricted residual, and its unknown is an error correction, not a replacement physical solution.
Core Conceptual Connection: Fine residual $\longrightarrow$ coarse error equation $\longrightarrow$ fine correction.

19.3 Experiments and Complexity

For Poisson with $N = 63, 127, 255$, use two weighted-Jacobi pre/post sweeps and a direct coarsest solve. Record reduction per cycle and work per unknown. Under ideal geometric coarsening, level sizes form a geometric series and a V-cycle costs $O(N)$. Mesh-independent convergence additionally requires smoothing and approximation properties.

Cost and Applicability Boundary layers, anisotropy, coefficient jumps, and complex geometry can defeat simple point smoothing or interpolation. Match line relaxation, semicoarsening, or improved coarse spaces to the operator.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/multilevel.py`

Ask AI

Ask AI: For seven fine interior nodes, write P, $P^T AP$, and the two-grid error matrix. Verify exact removal of coarse-space errors.
Follow-up: Plot mixed-frequency error after presmoothing, coarse correction, and postsmoothing. Compare V-cycle cost with additional fine sweeps.

Further Detail and Search Mesh-independent convergence proofs and full two-grid local Fourier analysis are omitted. See [5].

Keywords: coarse grid correction; Galerkin coarse operator; V cycle; W cycle; full multigrid

Chapter 20

Algebraic Multigrid

Chapter Overview

Geometric multigrid uses an existing mesh hierarchy; when the main input is a sparse matrix, coarse spaces must be built from algebraic coupling. Classical AMG and smoothed aggregation use strong connections, near-nullspace modes, and PDE information to construct interpolation. Both retain the error directions that smoothing leaves behind.

Learning Objectives Explain algebraically smooth error; distinguish classical interpolation from aggregation; record operator complexity and coarse costs.

Core Question Given only a sparse matrix, which errors should the coarse level represent?

20.1 From Geometric to Algebraic Smoothness

An error changed little by relaxation is algebraically smooth relative to that smoother. For typical diffusion M-matrices, large negative off-diagonal entries often signal strong coupling. One classical strength criterion is $-a_{ij} \geq \theta \max_{k \neq i}(-a_{ik})$. The criterion depends on signs and problem type and does not transfer unchanged to arbitrary nonsymmetric or block systems. Classical AMG selects coarse variables and builds interpolation through strong connections. Smoothed aggregation first groups variables into tentative interpolation P_0, then improves its energy properties using

$$P = (I - \omega D^{-1} A) P_0, \qquad A_c = P^T A P$$

or related operations. Elasticity often requires rigid-body near-null modes; more complex systems may require block information or dedicated strategies.

Potential Conceptual Jump

Potential Question: Why does AMG need physical information beyond the nonzero pattern?
Author Supplement: A pattern identifies coupling but not low-energy or poorly relaxed combinations. Coefficient magnitudes, signs, anisotropy, and near-nullspaces determine the directions that interpolation must retain.
Core Conceptual Connection: Matrix coupling $\longrightarrow$ difficult error modes $\longrightarrow$ appropriate interpolation space.

20.2 Complexity and Reliable Combinations

Grid and operator complexities are

$$C_g = \frac{\sum_\ell N_\ell}{N_0}, \qquad C_o = \frac{\sum_\ell \operatorname{nnz}(A_\ell)}{\operatorname{nnz}(A_0)}.$$

Denser coarse levels may increase setup, memory, and communication even when iteration counts decrease. AMG can be a standalone iteration or an outer preconditioner. Standard CG needs a fixed SPD overall cycle; variable inner strategies generally motivate FGMRES.

20.3 An Executable Teaching Model

The one-dimensional Poisson example combines adjacent aggregation, tentative constant interpolation, one Jacobi smoothing step, and an exact coarse solve into a two-level action. It verifies $P^T AP$, symmetry, and residual correction. Production AMG adds recursive coarsening, robust strength measures, truncation, and parallel coarse-level treatment; performance measurements refer to each complete implementation.[20]

Cost and Applicability Report setup, C_g, C_o, cycle cost, and outer iterations. Strongly nonsymmetric transport, saddle-point systems, and high-frequency indefinite waves often need specialized preconditioners; diffusion-AMG results do not directly generalize.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/multilevel.py`

Ask AI

Ask AI: For seven-point Poisson, construct adjacent-aggregate P_0, smoothed P, and $P^T AP$. Compare correction of constant and oscillatory errors.
Follow-up: Vary coefficient jumps and aggregation. Record complexity, setup, and PCG true residuals, checking symmetry and positive definiteness first.

Further Detail and Search Ruge–Stüben interpolation formulas and smoothed-aggregation convergence theory are omitted. See [3, 5, 20].

Keywords: algebraically smooth error; strength of connection; classical AMG; smoothed aggregation; near nullspace

Chapter 21

Solver Combinations and Engineering Selection

Chapter Overview

Factorization, Krylov projection, local corrections, and multilevel methods form the solver's building blocks. Matrix structure, tolerances, reuse, and hardware costs organize them into a complete workflow. The subsequent Roofline analysis extends evaluation from one operator call to the full computation required for a target error and physical final time.

Learning Objectives Choose compatible solver combinations; separate setup and solve; allocate discretization, linear, and nonlinear error budgets.

Core Question How are algorithmic components assembled into a verifiable, reusable, comparable solver?

21.1 Conditions for Combinations

System	Candidate combination	Required checks
SPD diffusion	CG+IC or symmetric MG/AMG	Fixed SPD inverse action; nullspace handling
Symmetric indefinite	MINRES+SPD block preconditioner	Symmetry and preconditioner definiteness
Nonsymmetric transport–diffusion	GMRES+ILU/AMG	Restart, nonnormality, true residual
Changing inner solves	FGMRES+variable MG/local solves	Actual correction directions and inner cost
Moderate size, multiple right-hand sides	Sparse direct or recycled Krylov	Setup amortization, memory, numerical rank

Each combination assumes the stated matrix structure; AMG additionally needs coarse spaces and smoothers adapted to the problem's persistent error modes.

21.2 Tolerances and Nonlinear Outer Iteration

A Newton step $J_k \delta \boldsymbol{u}_k = -\boldsymbol{F}(\boldsymbol{u}_k)$ may impose

$$\|J_k \delta \boldsymbol{u}_k + \boldsymbol{F}(\boldsymbol{u}_k)\| \leq \eta_k \|\boldsymbol{F}(\boldsymbol{u}_k)\| .$$

Use looser inner tolerances early and reduce η_k near the solution to match linear work to outer convergence needs; line searches or trust regions provide globalization. Discretization error and quantities of interest determine final tolerances, with software defaults serving as initial settings.

Potential Conceptual Jump

Potential Question: Why can more accurate linear solves fail to improve the full calculation?
Author Supplement: Early Newton Jacobians represent only local nonlinear behavior. Solving the linearized system more accurately does not remove linearization error. Evolution also has space and time errors. Allocate effort to the sources controlling the result.
Core Conceptual Connection: Overall error target $\longrightarrow$ layered tolerances $\longrightarrow$ total solution time.

21.3 A Minimal Selection Experiment

On the same Poisson and advection–diffusion systems, compare direct solves, unpreconditioned Krylov, ILU-GMRES, and two-level preconditioning. Check assumptions and final errors, then record setup, application, counts, and factor/hierarchy storage. Repeat for ten right-hand sides to examine reuse.

Cost and Applicability Total cost includes assembly, ordering, preconditioner setup, operator action, orthogonalization, communication, and final checks. Triangular solves, SpMV, dot products, and dense blocks have distinct hardware fingerprints, examined in Part V.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`sparse_linear_algebra/solvers.py;sparse_linear_algebra/multilevel.py`

Ask AI

Ask AI: Write selection records for two-dimensional Poisson and nonsymmetric advection–diffusion: structural evidence, assumptions, tolerance rationale, setup cost, and failure fallback.
Follow-up: Use measured setup/solve times to plot multiple-right-hand-side amortization. Distinguish fewest iterations from shortest total time.

Further Detail and Search Specialized block preconditioning for coupled PDEs and global nonlinear convergence proofs are omitted. See [3, 20].

Keywords: solver configuration; setup amortization; inexact Newton; tolerance budget; time to solution

Part V

Roofline-Driven High-Performance Computing

Chapter 22

The Roofline Performance Model

Chapter Overview

Having obtained a solution algorithm, we next identify runtime limits. Arithmetic intensity connects throughput with data movement, giving testable Roofline bounds. Later discretizations use the same analysis, provided work, memory level, and timing scope agree. Identify the limiting resource before choosing an optimization.

Learning Objectives Count FLOPs and bytes and locate the ridge point; distinguish theoretical peaks, sustained bandwidth, and measured throughput; compare GEMM and SpMV.

Core Question Why do floating-point kernels achieve widely different throughput?

22.1 Two Resource Bounds

For work F FLOPs, traffic Q bytes at a specified memory level, sustained bandwidth B bytes/s, and precision-specific peak P_{peak} FLOP/s,

$$I = F/Q, \quad P_{\text{attained}} = F/T, \qquad P_{\text{attained}} \le \min(P_{\text{peak}}, BI), \qquad I_{\text{ridge}} = P_{\text{peak}}/B.$$

Roofline plots arithmetic intensity against throughput to expose resource bounds.[8] The model organizes performance analysis; computation, storage, and communication limits predate it. Latency, dependencies, instruction mix, insufficient parallelism, and imbalance can place performance below the roof. Pipelines, vector units, multicore processors, and accelerators increase arithmetic throughput, while memory latency and bandwidth need not improve proportionally: the memory wall. Caches, blocking, and batching mitigate this imbalance through reuse. Machine balance P_{peak}/B sets the intensity needed to become compute-bound.[7] Peak utilization is $P_{\text{attained}}/P_{\text{peak}}$; Roofline-relative utilization is $P_{\text{attained}}/\min(P_{\text{peak}}, BI)$. Both require consistent precision, work, and traffic definitions and measure different efficiencies.

Potential Conceptual Jump

Potential Question: Why is counting additions and multiplications insufficient?
Author Supplement: The same arithmetic may reuse register data or wait for fresh operands. Intensity relates work to movement at a chosen memory level. Reducing FLOPs may leave limiting traffic unchanged, even lowering reported throughput while shortening runtime.
Core Conceptual Connection: Operator work $\longrightarrow$ data reuse $\longrightarrow$ limiting resources and runtime.

22.2 GEMM and CSR-SpMV

Dense square multiplication $C = AB$ costs approximately $2n^3$ FLOPs. Reading each input and writing the output once gives an ideal double-precision $24n^2$ bytes and intensity $n/12$. Cache capacity, blocking, and write allocation alter actual traffic. CSR-SpMV counts one multiply-add, or 2 FLOPs, per nonzero. Double-precision values and 32-bit column indices require at least 12 bytes, plus row pointers, $\boldsymbol{x}$ access, and output. Intensity generally remains low even with substantial $\boldsymbol{x}$ reuse. Index width, cache hits, and access patterns determine the specific byte model.

22.3 Models, Timing, and Calibration

The schematic uses assumed values $P_{\text{peak}} = 1000$ GFLOP/s and $B = 100$ GB/s, with a ridge at 10 FLOP/byte, to illustrate the roof geometry. CPU kernel plots instead use measured times and work models. Positioning those measurements against local hardware roofs requires a separate calibration. Calibration separately measures sustained bandwidth, precision-specific throughput,

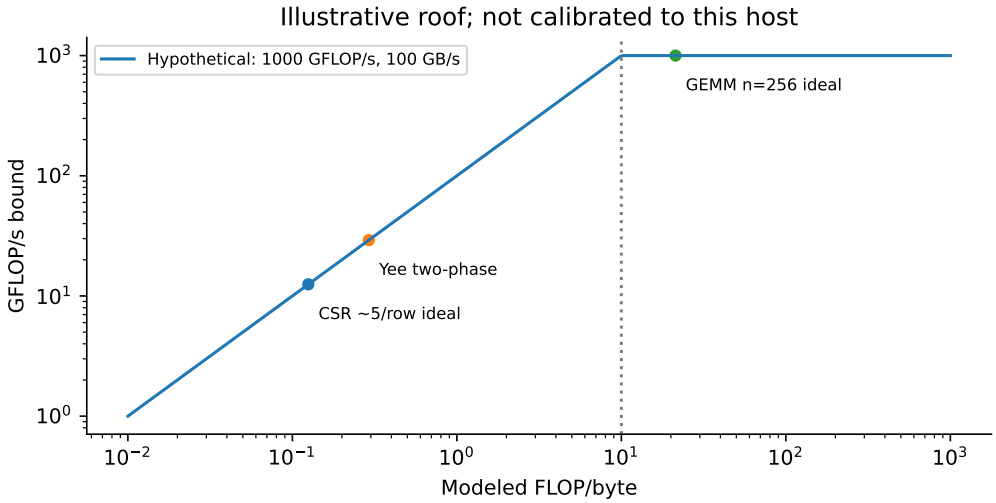


Figure 22.1: Roofline for hypothetical hardware; neither the roof nor the byte model is a DRAM-counter measurement.

and actual traffic. Different cache levels can have different roofs. DRAM models do not describe cache-resident small arrays.

Cost and Applicability Bandwidth-bound kernels benefit from reduced traffic, reuse, and layout improvements; compute-bound kernels benefit from vectorization and instruction efficiency. GFLOP/s diagnoses resource use, while total time at fixed error measures computational benefit.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`roofline/kernels.py`

Ask AI

Ask AI: For CSR with 32- and 64-bit indices, derive ideal-reuse and per-nonzero x-read traffic models. Compute intensities and identify omitted write-allocation effects.
Follow-up: Given GEMM/SpMV times and array sizes, determine whether they can share one DRAM Roofline and identify missing measurements.

Further Detail and Search Hardware-counter calibration and empirical hierarchical-Roofline tooling are omitted. See [8, 18].

Keywords: arithmetic intensity; operational intensity; ridge point; sustained bandwidth; hierarchical Roofline

Chapter 23

Computational and Memory Fingerprints of Discretizations

Chapter Overview

Roofline analysis begins with an operator's work and data traffic. Comparing finite differences, finite volumes, FEM, DG, FR, and LBM shows how degrees of freedom, locality, order, and geometry shape reuse. These computational and access patterns form performance signatures, made concrete by the following Yee update and integrated experiments.

Learning Objectives Construct work and access models; distinguish assembled and matrix-free operators; compare volume, face, streaming, and collision terms.

Core Question How does spatial discretization determine what a machine reads and computes?

23.1 Finite Differences and Finite Volumes

A three-dimensional seven-point Laplacian can be evaluated as

$$(\boldsymbol{L}u)_{ijk} = h^{-2}(u_{i+1,j,k} + u_{i-1,j,k} + u_{i,j+1,k} + u_{i,j-1,k} + u_{i,j,k+1} + u_{i,j,k-1} - 6u_{ijk})$$

Counting five neighbor additions, multiplication by 6, subtraction, and scaling gives 8 FLOPs. Seven reads and one write give 64 semantic bytes per double-precision output. Ideal spatial reuse approaches one read and one write, or 16 bytes, excluding write allocation. Spatial and temporal blocking improve within-step and cross-step reuse, subject to boundaries and dependencies. Finite volumes combine gradient reconstruction, limiters, face Riemann fluxes, and residual accumulation. Unstructured faces introduce gathers/scatters. Computing each face once avoids duplicate fluxes but may require coloring, atomics, or buffers; independent cell traversal avoids write conflicts but repeats flux work. Compare at equal accuracy and conservation requirements.

23.2 Finite Elements, DG, and FR

Low-order assembled FEM may be dominated by sparse multiplication, while assembly also involves quadrature, geometry, and sparse accumulation. Matrix-free evaluation recomputes element actions, trading arithmetic for entry and index traffic. For three-dimensional tensor elements with $n = p + 1$ points per direction, a dense volume operator can cost $O(n^6)$; sum factorization reduces directional derivatives to $O(n^4)$:

$$(\partial_\xi u)_{ijk} = \sum_{\ell=0}^{p} D_{i\ell} u_{\ell jk}.$$

Each direction uses n^3 length-n dot products, totaling $3n^3(2n-1)$ FLOPs. Reading the input once and writing three derivatives gives approximately $32n^3$ bytes of field traffic, plus derivative matrices, geometry, and materials. Batching, register pressure, and faces change actual reuse. DG volume terms, face fluxes, and liftings admit separate models; FR combines nodal derivatives with interface-flux corrections. Both exploit local matrix products on tensor grids. Correction functions, quadrature, and nodes determine their operator relationship, while energy or spectral estimates establish stability.

Potential Conceptual Jump

Potential Question: How can higher order be faster despite more operations per element?
Author Supplement: Higher order may improve reuse and reach a target error with fewer elements. It also changes unknown counts, face costs, and explicit stability limits. Evaluate spatial error, temporal error, and total step count together.
Core Conceptual Connection: High-order local work $\longrightarrow$ reuse and error changes $\longrightarrow$ total-cost comparison.

23.3 LBM Streaming and Collision

Each lattice site stores Q populations; BGK collision is

$$f_q^* = f_q - \tau^{-1}(f_q - f_q^{\mathrm{eq}}), \qquad f_q(\boldsymbol{x} + \boldsymbol{c}_q\Delta t, t + \Delta t) = f_q^*(\boldsymbol{x}, t).$$

D2Q9, D3Q19, and D3Q27 have 9, 19, and 27 populations. Two-buffer streaming requires at least $2Q$ scalar reads/writes; double-precision D2Q9 gives 144 bytes/site before collision temporaries and boundaries. SoA groups one population across sites; AoS groups populations at each site. Suitability depends on SIMD/SIMT mapping and collision implementation. Push writes postcollision values to neighbors; pull reads from neighbors. Injective periodic streaming avoids write conflicts, while complex boundaries require separate treatment.

23.4 A Common Comparison

Method	Reuse and access	Typical limitations
FDM	Regular neighbors and cross-point reuse	Bandwidth, boundaries, time steps
FVM	Face geometry, reconstruction, fluxes	Irregular access, branches, accumulation
FEM	Local quadrature or sparse products	Different assembly/application paths
DG	Tensor volume and face data	Registers, lifting, stable step size
FR	Point derivatives and flux corrections	Nodes, correction functions, face exchange
LBM	Population streaming and collision	Layout, storage, complex boundaries
FDTD	Staggered curl stencils	Two-phase dependencies, bandwidth, dispersion

Cost and Applicability The table uses access patterns to locate bottlenecks. Method comparisons fix the physical problem, error, precision, hardware, and parallel scale. For NumPy implementations, include temporary-array traffic in addition to the ideal fused-kernel model.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`roofline/kernels.py;dg_fr/transport.py;lbm/stream_collision.py`

Ask AI

Ask AI: List operations and reads/writes for seven-point Laplacian, tensor derivatives, and D2Q9 streaming. Identify cache-reusable reads.
Follow-up: Compare first-order upwind FVM and high-order DG for the same periodic sine transport, recording spatial error, steps, and total time. Keep LBM fluid physics distinct from arbitrary scalar transport.

Further Detail and Search General curved-element sum factorization and FR energy-stability proofs are omitted. See [10, 13, 14, 8].

Keywords: sum factorization; matrix-free FEM; DG volume surface; FR correction; LBM SoA AoS

Chapter 24

Yee-FDTD: Discrete Structure and Performance

Chapter Overview

Volume II established compatible discretizations of Maxwell evolution and constraints. Yee degrees of freedom, staggered time levels, and curl structure now turn topological identities into array updates and explicit work and traffic models. The discrete complex, phase relations, and two-phase dependencies define the constraints within which performance is optimized.

Learning Objectives Write Yee updates and constraints; distinguish incidence from material Hodge operators; calculate three traffic models and check dispersion and energy.

Core Question How can Yee bottlenecks be identified while preserving Maxwell structure?

24.1 Degrees of Freedom, Materials, and the Grid

For stationary, nondispersive isotropic media,

$$\varepsilon \partial_t \boldsymbol{E} = \nabla \times \boldsymbol{H} - \boldsymbol{j}, \qquad \partial_t \boldsymbol{B} = -\nabla \times \boldsymbol{E}, \qquad \boldsymbol{B} = \mu \boldsymbol{H}.$$

The coefficients ε, μ remain explicit in material and wave-speed relations. On a cubic grid, E_x lies at $(i + \frac{1}{2}, j, k)$, with E_y, E_z obtained cyclically; B_x lies at $(i, j + \frac{1}{2}, k + \frac{1}{2})$, with the others cyclic. E uses integer times and B half-integer times.

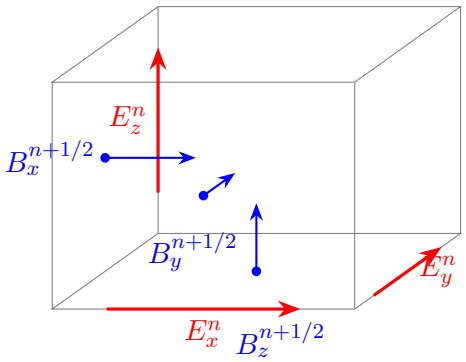


Figure 24.1: Representative edge electric and face magnetic components in a cubic cell. Arrows indicate component directions; not all edge/face unknowns are shown.

Point components are samples on a uniform grid. Integral unknowns are edge circulations $e_\ell = \int_\ell E$ and face fluxes $b_f = \int_f B$. Lengths, areas, and materials connect the representations; point arrays are not dimensionless incidence matrices.

24.2 Topology and Matrix-Free Updates

Incidence matrices G, C, D satisfy $CG = 0, DC = 0$. With compatible primal/dual grids and boundary pairing,

$$\dot{\boldsymbol{b}} = -C\boldsymbol{e}, \qquad M_\varepsilon \dot{\boldsymbol{e}} = C^T M_{\mu^{-1}} \boldsymbol{b} - \boldsymbol{j}_h.$$

C encodes orientation; $M_\varepsilon, M_{\mu^{-1}}$ encode metric and material data. Uniform orthogonal Yee grids admit diagonal material actions. Thus $\partial_t(D\boldsymbol{b}) = 0$. With dual-divergence convention $D_e = -G^T$, charge is $\boldsymbol{q} = D_e M_\varepsilon \boldsymbol{e}$ and continuity requires $\dot{\boldsymbol{q}} = -D_e \boldsymbol{j}_h$, with compatible boundary fluxes. Define forward differences δ_α^+ and backward differences δ_α^-. On a periodic uniform grid,

$$B_x^{n+1/2} = B_x^{n-1/2} - \Delta t(\delta_y^+ E_z^n - \delta_z^+ E_y^n),$$

$$E_x^{n+1} = E_x^n + \frac{\Delta t}{\varepsilon}[\delta_y^- H_z^{n+1/2} - \delta_z^- H_y^{n+1/2} - j_x^{n+1/2}],$$

with cyclic counterparts. This is a block-sparse action without CSR assembly. Explicit matrices remain useful for small spectral checks, implicit integration, and operator comparisons. The execution order is

```
for each time step:
exchange required E halos
B_half = B_half - dt * curl_forward(E)
exchange required B_half halos
E = E + dt * curl_backward(B_half)
```

Potential Conceptual Jump

Potential Question: Why can curl updates not be changed arbitrarily to in-place operations?

Author Supplement: Both curls must retain consistent time levels and spatial orientation. The magnetic half-step reads old electric fields; the electric step reads new magnetic fields. Overwriting unread values changes the operator, stability, dispersion, and constraints.

Core Conceptual Connection: Staggered unknowns $\longrightarrow$ two-phase dependency graph $\longrightarrow$ admissible fusion and synchronization.

24.3 Auditable Work and Traffic

For source-free, constant-coefficient vacuum with precomputed inverse spacings, each component uses two differences, two scalings, a subtraction, a Δt multiplication, and an update: 7 FLOPs, or 42 per six-component cell-step. An FMA counts as two FLOPs. This counts separately formed directional differences. On an isotropic equal-spacing grid, precombining $\Delta t/h$ gives three subtractions, one scaling, and one update per component: 5 FLOPs, or 30 per cell-step. The CUDA/MPI code uses this form. Count the implemented work; do not assign 42 FLOPs to the fused kernel. For double precision, excluding materials and sources:

Model	bytes/cell-step	I (FLOP/byte)
Four neighbors+self read+write	288	42/288
Ideal reuse within each phase	144	42/144
Six fields resident for full step	96	42/96

The three values describe traffic under specified reuse assumptions. Actual DRAM traffic also depends on caches, write allocation, temporaries, boundaries, and coefficient access. For large two-phase arrays, working-set size determines whether whole-step residency is feasible. Six SoA arrays support contiguous spatial access by neighboring threads. GPU execution can use separate B/E launches, further split by component if needed. Registers, launch cost, and global synchronization determine the tradeoff. Shared memory helps only when it reduces actual repeated movement without unduly reducing parallelism.

24.4 Stability, Dispersion, and Acceptance

With vacuum wave speed $c = (\varepsilon\mu)^{-1/2}$, the uniform three-dimensional condition is

$$c^2\Delta t^2(h_x^{-2} + h_y^{-2} + h_z^{-2}) \le 1.$$

Periodic Fourier modes satisfy

$$\frac{\sin^2(\omega_h \Delta t/2)}{c^2\Delta t^2} = \sum_\alpha \frac{\sin^2(k_\alpha h_\alpha/2)}{h_\alpha^2}.$$

CFL controls numerical growth, while the dispersion relation describes phase propagation. Evaluate fixed-distance propagation errors together with direction, resolution, and total step count. For normalized periodic vacuum, an exact-arithmetic invariant is

$$\mathcal{H}^n = \frac{1}{2}\,\|E^n\|^2 + \frac{1}{2}\langle B^{n-1/2}, B^{n+1/2}\rangle$$

with the appropriate volume weights. This is a modified staggered energy, not a sum of squares at unrelated times. Its relation to physical energy under stable time steps must be specified.

Cost and Applicability Record initial phase, final physical time, error, $\mathrm{div}_h B$, charge residual, modified energy, and advance time. Include nontrivial random-field curl–divergence checks, not just a transverse wave. Periodic lossless modes have unit-modulus amplification in the stable range; boundaries, material losses, and filtering may add dissipation and require separate records.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`fdtd/yee.py`

Ask AI

Ask AI: Derive neighboring indices for B_x and E_x from their locations. Verify $DC = 0$ term by term and identify the length/area factors.
Follow-up: Derive Yee dispersion and CFL from Fourier updates. Compare modified staggered energy with sums of squares at different times.

Connections Volume II explains geometry and constraints; the material matrices, traffic, and time levels here support Chapters 25, 28, and 29.

Further Detail and Search PML, dispersive materials, curvilinear Hodge operators, and PEC details are omitted. See [9, 18].

Keywords: Yee staggered grid; incidence matrix; discrete Hodge; numerical dispersion; modified energy

Chapter 25

Time Integration and Total Solution Cost

Chapter Overview

Spatial throughput gives the cost of one call; time integration determines call counts and implicit-solve work. A common framework connects RK, staggered integration, ADER, CERK, LTS, and IMEX to stability, phase accuracy, and structure preservation. At a specified error and final time, step size, stages, solves, and communication jointly determine total cost.

Learning Objectives Relate semidiscrete spectra to stability regions; distinguish symplecticity from energy conservation; analyze high-order, local-step, and stiff-splitting costs.

Core Question Why can a simulation remain slow when one spatial evaluation is fast?

25.1 Method of Lines and Stability Regions

Spatial discretization gives $M\dot{\boldsymbol{u}} = \boldsymbol{R}(\boldsymbol{u})$. Each explicit RK stage applies $M^{-1}\boldsymbol{R}$, cheaply for local or diagonal mass matrices. For $\dot{y} = \lambda y$, one step is $y^{n+1} = R(z)y^n$ with $z = \Delta t\lambda$; absolute stability requires $|R(z)| \leq 1$. Classical RK4 has $R(z) = 1+z+z^2/2+z^3/6+z^4/24$. Spectra, nonnormality, and energy estimates jointly constrain usable steps. For purely imaginary λ, forward Euler has $|1 + \mathrm{i}\omega\Delta t| > 1$, producing modal growth. The time integrator's stability region is therefore part of the complete energy analysis.

25.2 Staggering, Symplecticity, and Phase

Yee/leapfrog staggers suitable split systems and is conditionally stable. Störmer–Verlet is second-order symplectic for separable Hamiltonian systems. Implicit midpoint

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^n}{\Delta t} = A\frac{\boldsymbol{u}^{n+1} + \boldsymbol{u}^n}{2}$$

exactly preserves a quadratic energy when $A^TQ + QA = 0$ with fixed SPD Q. It is symplectic for general Hamiltonian systems, but symplecticity does not generally imply exact preservation of the original Hamiltonian. For $\lambda = \mathrm{i}\omega$, midpoint has phase frequency $\omega_h = 2\arctan(\omega\Delta t/2)/\Delta t$. Stable amplitude can accompany accumulated phase error; inaccurate linear solves also affect preservation.

25.3 ADER, CERK, and LTS

ADER constructs high-order time representations using element-local space–time prediction, followed by interface space–time flux correction. Prediction can improve local reuse; cost, stability, and order depend on the implementation. CERK denotes continuous explicit Runge–Kutta, adding an within-step approximation

$$\boldsymbol{u}(t_n + \theta\Delta t) \approx \boldsymbol{u}_n + \Delta t \sum_i b_i(\theta)\boldsymbol{k}_i, \quad 0 \le \theta \le 1.$$

Within-step accuracy follows the continuous-extension order conditions; endpoint accuracy follows those of the RK update. Coupling and output therefore check the two orders separately. LTS groups elements by size or wave speed. Conservation requires one shared time-integrated interface flux, with consistent fine-step accumulation and coarse correction. Interpolation, flux registers, scheduling, and imbalance add cost.

25.4 IMEX and Stiff Sources

Split $\dot{\boldsymbol{u}} = F(\boldsymbol{u}) + G(\boldsymbol{u})$, treating propagation explicitly and stiff sources implicitly. Additive RK stages are

$$U_i = \boldsymbol{u}^n + \Delta t \sum_{j<i} \widetilde{a}_{ij} F(U_j) + \Delta t \sum_{j \le i} a_{ij} G(U_j),$$

The endpoint uses separate explicit and implicit weights. Order and stability depend on the paired tableaux and coupling conditions. Conductive Maxwell may use $G_E = -\sigma E/\varepsilon$. A first-order split example is

$$E^{n+1} = \frac{E^n + \Delta t\,(\operatorname{curl} H)^n/\varepsilon}{1 + \Delta t\,\sigma/\varepsilon}.$$

This illustrates cheap local implicit damping. If propagation uses forward Euler, the $\sigma = 0$ wave limit remains unsuitable. Choose an IMEX pair compatible with the hyperbolic explicit stability region. Global diffusion or coupled implicit sources may reintroduce large linear systems.

Potential Conceptual Jump

Potential Question: Why do high-order or implicit methods not automatically reduce cost?
Author Supplement: Higher order changes stage counts and error constants; implicit treatment changes stability limits but adds solves. Local steps change scheduling and communication. At equal error requirements, determine usable steps, total steps, and the complete per-step execution graph together.
Core Conceptual Connection: Stability and accuracy $\longrightarrow$ usable time step $\longrightarrow$ stages, solves, and communication.

25.5 Minimal Comparisons

For periodic sine transport, compare RK4, second-order methods, and staggered wave updates. For $\dot{y} = \mathrm{i}\omega y - \sigma y$, compare explicit/IMEX stability and phase. Assess error at a fixed final time rather than only plotting $|R|$.

Cost and Applicability $T_{\text{total}} = T_{\text{setup}} + N_t T_{\text{step}} + T_{\text{output}}$, where T_{step} includes all stages, implicit solves, and communication. ADER and LTS trade extra local work for opportunities to reduce global updates. CERK supplies time-aligned states; interface fluxes and the combined stability region control conservation and stability.

Ask AI

Ask AI: Derive midpoint's phase frequency from its amplification factor. Quantify phase error after one hundred periods and compare RK4.
Follow-up: Write shared flux integrals and fine-step accumulation for adjacent unequal-step elements. Explain why interpolated boundary values alone do not ensure conservation.

Further Detail and Search ADER predictors, CERK order conditions, and high-order IMEX tableaux are omitted. See Volume II's time-integration chapter and [10, 15, 16, 17].

Keywords: method of lines; symplectic integrator; ADER; continuous explicit Runge Kutta; conservative LTS; IMEX additive RK

Chapter 26

Memory Hierarchies and Operator Implementation

Chapter Overview

Ideal traffic estimates set reuse targets; implementation determines the storage levels through which data actually move. Registers, caches, and DRAM connect to layout, blocking, fusion, and SIMD through temporaries, dependencies, and access paths. Optimization reduces movement at the limiting level and is evaluated under common accuracy and timing conventions.

Learning Objectives Analyze strides and cache reuse; explain fusion, blocking, SIMD, and matrix-free tradeoffs; design fair timings.

Core Question Why does a program repeatedly move data that ideally need only one read?

26.1 Hierarchy, Locality, and Layout

Registers are close to execution units; L1/L2/L3 caches provide increasing capacity, and DRAM supplies main capacity. Sharing arrangements depend on architecture. Contiguous access improves cache-line use and prefetching; large strides and random gathers may waste transactions. Working-set size determines which reuse survives. In C-order three-dimensional arrays, the final index is contiguous. Align inner loops and thread mappings accordingly. SoA often favors cross-point vectorization; AoS may favor using every component at a point; AoSoA combines these within blocks. Judge layouts by the actual kernel.

26.2 Fusion and Blocking

Evaluating $w = au + bv$ through two temporary arrays adds writes and reads. Fusion can read u, v and write w directly, but increases live registers; excessive fusion can cause spills. Spatial blocking retains neighboring stencil planes. Temporal blocking reuses subdomains across steps while respecting dependence cones and expanded halos. Two-phase Yee or multistage RK cannot be fused arbitrarily across steps without synchronization. Tensor DG can reuse derivative matrices and batch elements. Keeping three derivatives and fluxes simultaneously may exceed register/cache capacity. Matrix-free evaluation reduces sparse indexing but repeats geometry work; mixed precision additionally requires error analysis.

Potential Conceptual Jump

Potential Question: Why can zero-copy or matrix-free code remain slow?
Author Supplement: Avoiding explicit copies or matrix entries removes only one class of work. Cache misses, hidden temporaries, migration, and extra arithmetic may still dominate. Infer traffic from actual access paths, not interface names.
Core Conceptual Connection: Storage representation $\longrightarrow$ actual access paths $\longrightarrow$ per-level resource use.

26.3 SIMD and Minimal Experiments

SIMD applies one instruction to several elements, requiring independent iterations, suitable layout/alignment, and manageable branching. Vectorization reports, assembly, and counters are more reliable than source appearance. Compare contiguous and strided seven-point traversal, D2Q9 SoA/AoS, and explicit-Kronecker versus sequential tensor contractions. Warm up, retain outputs, repeat timings, and report statistics. Separate large arrays from cache-resident cases.

Cost and Applicability Report allocation and kernel times separately, comparing preallocated and dynamically allocated implementations within a common timing scope. Individual NumPy operations invoke compiled kernels; fusion across operations depends on the execution strategy.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`roofline/kernels.py;lbm/stream_collision.py;dg_fr/transport.py`

Ask AI

Ask AI: List every NumPy seven-point temporary and its lifetime. Compare resulting bytes with Chapter 23's fused lower bound.
Follow-up: Design working sets fitting L1, fitting L3, and exceeding last-level cache. Explain why one undifferentiated bandwidth cannot describe them all.

Further Detail and Search Cache replacement, compiler vectorization, and microarchitectural port analysis are omitted. See [8, 18].

Keywords: cache blocking; loop fusion; temporal blocking; SIMD; AoSoA; working set

Chapter 27

Parallel Throughput on CPUs and GPUs

Chapter Overview

Layout controls a single execution stream; parallel performance also requires sufficient independent work and resources. This chapter connects CPU threads and GPU SIMT to occupancy, coalescing, and device residency. Evaluate throughput, dependencies, launch, and migration together, using end-to-end timing to assess acceleration.

Learning Objectives Explain NUMA and false sharing; distinguish SIMD/SIMT; assess coalescing, occupancy, and transfer amortization.

Core Question Why do more threads or higher occupancy not necessarily shorten runtime?

27.1 CPU Threads and Memory Placement

Multicore execution distributes elements, rows, or grid blocks among threads. OpenMP loops need conflict-free writes or synchronization. Binding and first touch determine NUMA page placement; cross-socket access changes latency and bandwidth. Threads writing distinct variables on one cache line can still cause false sharing. Blocking and padding separate hotspots. SIMD and multicore threading are complementary levels. On hybrid-core processors, thread performance, affinity, and power state affect short experiments. The records therefore include CPU model, available logical processors, and BLAS configuration; sustained all-core throughput requires calibration with a corresponding workload.

27.2 GPU Execution Resources

CUDA groups threads into blocks, schedules blocks onto SMs, and executes warps in SIMT fashion.[18] Contiguous threads accessing contiguous elements favor coalescing. Divergence, irregular gathers, and insufficient parallelism reduce utilization. Registers and shared memory limit resident blocks/warps. Occupancy measures residency, not performance; raising it while sacrificing reuse or causing spills may be counterproductive. SpMV short and long rows have different workloads. DG elements can be batched into small matrix kernels. Yee/LBM should first align spatial indexing, then consider shared caching. Fixed launch overhead can dominate small problems.

27.3 Transfers, Asynchrony, and Residency

$$T_{\text{accelerator}} = T_{\text{H2D}} + T_{\text{kernels}} + T_{\text{D2H}} + T_{\text{synchronization}}.$$

Keep evolving fields, geometry, and preconditioner data on the device when possible. Copy/compute overlap requires hardware support, appropriate streams, and buffer management. An asynchronous call's return is not completion. Use device events or synchronization, separating first launch, steady kernels, and transfer-inclusive totals. Unified memory supplies shared addressing and demand migration, not the elimination of data movement. Faults, granularity, and residency still matter. Compare explicit prefetch and ownership policies with explicit-transfer measurements.

Potential Conceptual Jump

Potential Question: Why can timing an asynchronous call with a CPU clock produce a fictitious speedup?
Author Supplement: The host often only enqueues work. Stop timing after the corresponding device work completes, and state whether transfer and initialization are included.
Core Conceptual Connection: Host submission $\longrightarrow$ device execution $\longrightarrow$ completion synchronization and valid timing.

27.4 A Reproducible Interface

The CUDA program compares single-component AoS/SoA access, advances periodic Yee fields in two phases, and checks results on the host. Compilation, device availability, and numerical validation are recorded separately; timings are produced after device execution. A complete experiment also records model, driver, precision, block configuration, registers, device bandwidth, and transfer scope.

Cost and Applicability GPU benefits depend on size and residency. Sparse triangular dependencies, reductions, and small kernels may limit acceleration. CPU implementations should also avoid oversubscription between BLAS and outer threads.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`gpu/yee_layout.cu`

Ask AI

Ask AI: List warp addresses for SoA and AoS E_x updates. Estimate transaction use and identify quantities requiring profiler confirmation.
Follow-up: Design timings including transfers, warmup, repetitions, and synchronization. Derive how many steps amortize one device migration.

Further Detail and Search Device-specific pipelines and autotuning are omitted. See [18].

Keywords: OpenMP; NUMA first touch; false sharing; SIMT; coalescing; occupancy; device residency

Chapter 28

Distributed Computing and Communication Cost

Chapter Overview

As single-node computation accelerates, halo exchange, global dot products, and coarse-level communication enter the critical path. Message latency and volume complement compute and memory Roofline analysis, explaining scaling, progress, and overlap. Operator dependencies identify communication that can be hidden; timing establishes whether that overlap is realized.

Learning Objectives Build an alpha–beta model; distinguish halos and allreduce; explain overlap, pipelining, and coarse-level bottlenecks.

Core Question Why does speedup saturate even when local computation is fast?

28.1 Communication and Scaling

A communication phase may be approximated by

$$T_{\text{comm}} \approx n_{\text{msg}}\alpha + V/B_{\text{net}},$$

where α is effective message-start latency, V the explicitly defined volume, and B_{net} effective bandwidth. Protocol, topology, congestion, and concurrency affect parameters. Nonoverlapped costs add; ideal overlap suggests a more optimistic lower bound:

$$T \gtrsim \max(F/P_{\text{peak}}, Q/B_{\text{mem}}, T_{\text{comm}}).$$

This estimate describes a resource lower bound under ideal overlap; communication progress, contention, and dependency chains determine the realized time. Strong scaling fixes global size. As local cubic side ℓ decreases, volume work is $O(\ell^3)$ and face exchange $O(\ell^2)$, increasing surface-to-volume ratio. Weak scaling fixes local work but still encounters global reductions, network effects, and coarse levels.

28.2 Halos, Reductions, and Overlap

Regular stencils exchange neighboring faces; FVM/DG exchange face values or traces, with higher order increasing face data and local work. Yee exchanges follow magnetic/electric phase dependencies. Krylov adds dot-product allreduces to operator halos. Short CG recurrences retain global synchronization; GMRES orthogonalization can add reductions. Post nonblocking receives/sends,

compute halo-independent interiors, wait, then update boundaries. Actual progress and overlap depend on MPI and runtime configuration and require measurement.[19] Pipelined Krylov overlaps reductions with computation. Communication-avoiding block or s-step schemes reduce synchronization but may increase basis ill-conditioning, work, and residual drift. Numerical stability requires simultaneous checks.

Potential Conceptual Jump

Potential Question: Why can equal communication volumes have different costs?
Author Supplement: Many small messages are latency-sensitive; fewer large messages are more bandwidth-sensitive. Critical-path placement, progress, and contention also differ. Bytes alone do not describe these dependencies.
Core Conceptual Connection: Message counts and sizes $\longrightarrow$ progress and dependencies $\longrightarrow$ critical-path time.

28.3 A Minimal MPI Experiment

The program partitions a three-dimensional periodic field in one direction, exchanges faces using `Sendrecv` or `Irecv/Isend+Waitall`, and checks the seven-point operator on a global Fourier mode. Records include ranks, local size, maximum-rank time, and error. Single-host runs verify MPI semantics; cross-node runs measure network scaling. Multidimensional partitioning additionally requires six-face exchange and a partition strategy.

Cost and Applicability AMG coarse levels have shrinking local work and increasing communication ratios, sometimes requiring process aggregation. Time operators, preconditioners, reductions, and I/O separately. Strong-scaling comparisons use the slowest rank.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`mpi/halo.cpp`

Ask AI

Ask AI: For an n^3 grid split into p approximately cubic subdomains, derive local work, six-face halo bytes, and surface-to-volume ratio.
Follow-up: Draw a nonblocking halo event graph, marking overlappable interiors and dependent boundaries. Design a test of actual progress.

Further Detail and Search Network-topology models, pipelined-Krylov error analysis, and cross-node tuning are omitted. See [19, 20].

Keywords: alpha beta model; strong scaling; weak scaling; halo exchange; allreduce; pipelined Krylov; MPI progress

Chapter 29

Integrated Cases Across the Volumes

Chapter Overview

The preceding chapters separated discrete operators, algebraic algorithms, and execution costs. Reproducible experiments now reconnect them: first verify the model, errors, and constraints, then record solution and data-movement costs. Comparing total time for a common problem and target error turns kernel analysis into an integrated computational decision.

Learning Objectives Create consistent experiment records; complete Maxwell and high-order transport cases; distinguish kernel benchmarks, complete solves, and cross-model comparisons.

Core Question How can we establish that a faster implementation still solves the same problem?

29.1 Krylov and AMG: From Poisson to Preconditioning

Start with Volume I's two-dimensional Dirichlet Poisson discretization. Manufacture algebraic data to check direct, CG, and preconditioned true residuals; then use a continuous exact solution to assess discretization error. Record unknowns, nnz, ordering, fill, setup, iterations, time, and error. Use the algebraically manufactured right-hand side $\boldsymbol{b} = A\boldsymbol{x}_*$ to check the linear solver, then a continuous analytic solution to test PDE convergence. The code records these error levels separately. Multiple right-hand sides reuse the same factors to expose setup amortization, while the two-level AMG example verifies coarse-space correction. Separate Krylov SpMV, preconditioning, dot products, and orthogonalization. Record AMG smoothing, restriction, coarse solve, and interpolation by level. At small sizes, fewer iterations may not offset setup and Python overhead; at large sizes, coarse communication and reductions can change the comparison. Serial teaching hierarchies do not measure distributed AMG scaling.

29.2 Complete Maxwell Evolution

On the periodic unit cube with vacuum $c = 1$, use the right-going transverse wave

$$E_y(x,t) = \sin(2\pi(x-t)), \qquad B_z(x,t) = \sin(2\pi(x-t)).$$

All other components vanish. Initialize the magnetic field at its own spatial locations and $t = -\Delta t/2$, choosing $\Delta t \le \nu h$, $\nu < 1/\sqrt{3}$, and adjusting the step to reach the same T exactly. Compare L^2 error, phase, steps, and time on two grids. Half-time initial data must not be treated as full-time values. The CPU code also computes curls of random edge fields, checking

divergence and monitoring $\mathrm{div}_h B$ and modified energy. The source-free electric-divergence test represents zero charge; sources require compatible current and continuity updates. GPU execution

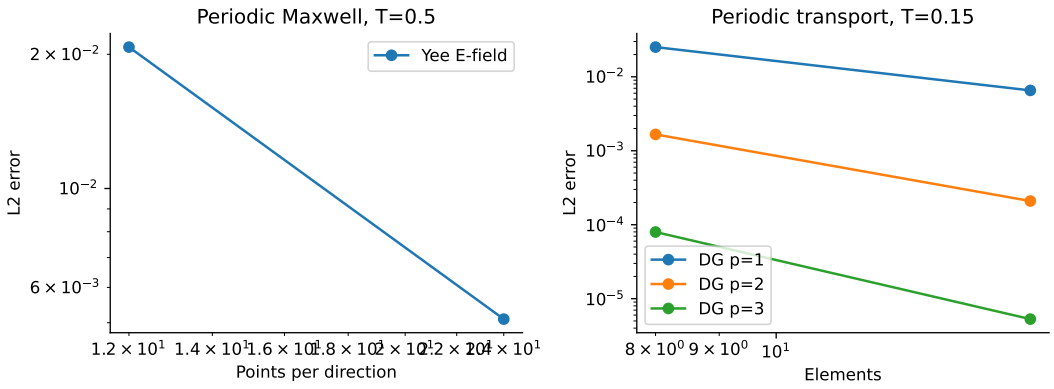


Figure 29.1: Measured CPU convergence data for Yee transverse waves and periodic transport. Error definitions and parameters accompany the records.

preserves SoA, phase order, and initial data. MPI exchanges required fields per phase before interior/boundary work. The CUDA full-field program provides a device path; the MPI solver includes phase halos and global constraints. The appendix and data record execution scope; cross-node and multi-GPU coupling can then be measured under the same error and constraint conventions.

29.3 Complete High-Order DG and the FR Interface

For $u_t + u_x = 0$ with periodic $u(x,0) = \sin(2\pi x)$, use elementwise Legendre modes $u_h = \sum_{m=0}^{p} \widehat{u}_m P_m(\xi)$. The weak form is

$$\frac{h}{2} M \dot{\widehat{\boldsymbol{u}}} = S\widehat{\boldsymbol{u}} - \boldsymbol{\phi}(1)\widehat{f}_R + \boldsymbol{\phi}(-1)\widehat{f}_L, \quad M_{mm} = \frac{2}{2m+1}, \quad S_{m\ell} = \int_{-1}^{1} P'_m P_\ell.$$

For positive speed, upwinding takes the left trace. Volume terms, face fluxes, and diagonal mass inversion are directly implementable. RK4 with conservative steps compares $p = 1, 2, 3$ and element counts in error, unknowns, steps, and time. At the two $p = 1$ Lobatto endpoints, the FR test uses $g_L = (P_2 - P_1)/2$, $g_R = (P_2 + P_1)/2$ Construct derivative corrections and verify equivalence with nodal DG under the same exact-mass and quadrature conventions. For other correction functions, compare their lifting and volume operators directly. Three-dimensional tensor derivatives are also checked against dense Kronecker application to verify sum factorization. MPI partitions elements, exchanges upwind traces at every RK stage, and retains local mass inversion. GPU optimization can batch same-order elements, reuse derivative matrices, and separate volume/face work. Increasing order raises register demand and volume-to-face ratio. Executed GPU checks currently cover Yee and layout; a complete DG GPU port is a separate extension.

Potential Conceptual Jump

Potential Question: Why can higher kernel throughput accompany a slower complete DG run?
Author Supplement: High-order local work increases intensity but also changes stable steps, stages, and error. Local throughput becomes an end-to-end benefit only through the element and step counts required for target accuracy.
Core Conceptual Connection: Kernel reuse $\longrightarrow$ accuracy and stable step $\longrightarrow$ cost to physical final time.

29.4 Access Comparisons: FDM, FVM, FEM, and LBM

First-order upwind FVM solves the same periodic transport problem as DG, enabling error and total-time comparisons. Seven-point FDM, Yee, DG tensors, LBM streaming and collision, and CSR/GEMM expose different access patterns. Cross-model update rates describe implementation signatures; method selection uses a common physical problem and target error. FEM checks assembled against local matrix-free actions. FVM/DG checks shared-flux conservation; LBM checks periodic mass. Accept correctness before timing.

29.4.1 Regular and Staggered Stencils

Seven-point FDM outputs one scalar field per application; Yee updates six staggered components in two phases. Both support contiguous indexing and halos, but equal site counts imply neither equal unknowns nor equal physics. Distinguish the 8-FLOP Laplacian, 42-FLOP difference-based Yee, and 30-FLOP fused Yee, stating traffic assumptions.

29.4.2 Adjacency, Irregular Access, and Geometry

The FVM case renumbers nonuniform periodic cells, gathers states through left/right face indices, computes fluxes, and accumulates residuals. Verify permutation equivalence and zero volume-weighted residual sum. This isolates numbering and gather/scatter. General unstructured meshes additionally require normals, areas, reconstruction neighborhoods, and graph partitioning. MPI exchanges follow partition-crossing faces, not a presumed fixed array-neighbor pattern.

29.4.3 Local Matrices and Population Layout

One-dimensional P1 FEM compares CSR with elementwise difference accumulation under identical boundary elimination. LBM separately times streaming, collision, and complete steps. Streaming tests check push/pull and AoS/SoA equality; periodic BGK checks mass. Collision live variables and boundary branches also influence GPU occupancy, which a CPU layout test alone cannot predict.

The Eight Case Interfaces FDM–Yee stencils, FVM adjacency, DG/FR volume/face structure, LBM streaming/collision, Krylov, AMG, complete Maxwell, and complete high-order DG cover eight performance interfaces. Code and appendices specify acceptance criteria and execution scopes.

29.5 Common Acceptance and Selection

Dimension	Required record
Physics/discretization	Equations, boundaries, initial data, T, mesh, order, unknowns
Accuracy	Space/time error, phase, true residual, conservation, constraints
Algorithmic work	Steps, stages, iterations, preconditioning, hierarchy complexity
Resources	Array/factor storage, modeled FLOPs/traffic, messages, reductions
Timing scope	Inclusion of setup, advance, transfer, communication, checks, output

Cost and Applicability Fixed-unknown comparisons analyze implementations; fixed-error comparisons select methods. Report both. Small CPU tests validate the workflow; large-scale optimization requires representative working sets, parallel scale, and counters.

Computational Example Shared companion code in `Vol0_Linear_Algebra_HPC/code_examples/`:
`run_all.py;run_accelerators.py;fdtd/yee.py;dg_fr/transport.py`

Ask AI

Ask AI: Run the experiments and plot error against time and unknown count from JSON. Check that physics, final time, and error norm agree within each comparison.
Follow-up: Choose an error threshold and search mesh, order, and time step jointly for Yee or DG, including setup, communication, and output in the budget.

Further Detail and Search Cross-node benchmarks and engineering-scale complex boundaries are further experiments. The experiment guide supplies reproduction commands and timing conventions. References: [9, 10, 8].

Keywords: verification versus validation; manufactured solution; error versus cost; constraint preservation; time to solution

Appendix A

Reproduction and Measurement Conventions

From the repository root, run `make vol0-examples` to generate CPU experiments, figures, and JSON; use `make vol0-en` to build this edition. Dependencies are Python, NumPy, SciPy, Matplotlib, and XeLaTeX. BLAS defaults to one thread, keeping thread configuration separate from algorithm comparisons.

Shared `results/measurements.json` records environment, parameters, timing, residuals, and errors; `results/validation.json` records numerical assertions. Byte counts use explicit work models. A null `dram_bytes_measured` means no hardware counters were used; effective bandwidth inferred from modeled bytes is not measured DRAM bandwidth.

GPU and MPI sources are under `gpu/` and `mpi/` in the shared code directory. Experiments record compilation, numerical device validation, and cross-node performance as separate evidence levels. Commands and timing conventions appear in `docs/experiments.md`. Figures come from the code; timings refer to the recorded implementation, workload, and hardware configuration.

A.1 A Common Error and Cost Record

Record equations, boundaries, initial data, physical final time, mesh/order, unknowns, step size/count, precision, error norms, constraints, setup, advance time, hardware, and threads. Fixed-grid throughput and fixed-error total time answer different questions.

A.2 Transferability

Recalibrate after changes to processor, power mode, threads, compiler, or libraries. Cache-resident arrays use roofs for the corresponding cache level; implementations with NumPy temporaries include intermediate reads and writes in their traffic model. Distributed experiments additionally record network, binding, message size, and reduction strategy.

A.3 Recorded Numerical Results

These tables are generated from the shared JSON records, not from new runs. CPU: Intel Core Ultra 7 155H; NumPy 1.26.4; SciPy 1.15.3. Double precision and single-thread BLAS. Times describe these implementations and small workloads.

Case	Cells/elements	p	Steps	L^2 error	Advance/s
Yee	12	–	14	2.076e-02	1.424e-03
Yee	24	–	27	5.094e-03	8.223e-03
DG	8	1	44	2.523e-02	2.015e-03
DG	16	1	87	6.550e-03	4.003e-03
DG	8	2	120	1.665e-03	5.416e-03
DG	16	2	240	2.091e-04	1.100e-02
DG	8	3	236	7.970e-05	1.056e-02
DG	16	3	471	5.295e-06	2.169e-02
FVM	32	0	12	5.534e-02	6.787e-05
FVM	64	0	24	2.786e-02	1.272e-04

Yee uses $T = 0.5$; DG/FVM use $T = 0.15$. DG and FVM errors compare reconstructed fields against exact fields by integration. FVM cell-average error is also stored. Errors and times for different equations are interpreted separately.

A.4 GPU and MPI Records

GPU kernels use constant-coefficient fusion: 30 FLOPs per cell-step. Device advance and transfer-inclusive times are separate. Both grids use 30 steps but different final times; their error ratio is not a convergence-order estimate. Remote GPU timings are short shared-load validation runs.

Device	n	T	Kernel/s	With transfers/s	CPU max. difference
Local RTX 500 Ada	32	0.3750	5.591e-04	1.427e-03	3.37e-16
Local RTX 500 Ada	64	0.1875	3.622e-03	1.015e-02	4.44e-16
GK RTX 4090	32	0.3750	1.745e-04	8.856e-04	3.37e-16
GK RTX 4090	64	0.1875	4.792e-04	5.303e-03	4.44e-16

MPI runs use one host, include halos, and do not measure cross-node scaling.

Host	Case	Cells/elements	Ranks	L^2 error	Max. rank time/s
Local	Yee	12	2	2.076e-02	2.516e-04
Local	Yee	24	2	5.094e-03	3.299e-03
Local	DG($p = 3$)	8	2	7.970e-05	4.386e-04
Local	DG($p = 3$)	16	2	5.295e-06	7.949e-04
GK	Yee	12	2	2.076e-02	1.304e-04
GK	Yee	24	2	5.094e-03	1.768e-03
GK	DG($p = 3$)	8	2	7.970e-05	2.962e-04
GK	DG($p = 3$)	16	2	5.295e-06	6.748e-04

The hardware JSON files retain one-/two-rank records, conservation, and constraints. No DRAM counters or calibrated roofs were used; semantic traffic and ideal reuse are separate models in the main text.

Bibliography

[1] Golub G H, Van Loan C F. *Matrix Computations*. 4th ed. Johns Hopkins University Press, 2013.

[2] Trefethen L N, Bau D. *Numerical Linear Algebra*. SIAM, 1997.

[3] Saad Y. *Iterative Methods for Sparse Linear Systems*. 2nd ed. SIAM, 2003. https://www-users.cse.umn.edu/~saad/IterMethBook_2ndEd.pdf.

[4] Barrett R, et al. *Templates for the Solution of Linear Systems*. SIAM, 1994. https://www.netlib.org/templates/templates.pdf.

[5] Briggs W L, Henson V E, McCormick S F. *A Multigrid Tutorial*. 2nd ed. SIAM, 2000.

[6] Trottenberg U, Oosterlee C W, Schüller A. *Multigrid*. Academic Press, 2001.

[7] Hennessy J L, Patterson D A. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2017.

[8] Williams S, Waterman A, Patterson D. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 2009, 52(4):65–76. DOI:10.1145/1498765.1498785. Original technical report: https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html.

[9] Yee K S. Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media. *IEEE Transactions on Antennas and Propagation*, 1966, 14(3):302–307. DOI:10.1109/TAP.1966.1138693.

[10] Hesthaven J S, Warburton T. *Nodal Discontinuous Galerkin Methods*. Springer, 2008.

[11] Bossavit A. *Computational Electromagnetism: Variational Formulations, Complementarity, Edge Elements*. Academic Press, 1998.

[12] Monk P. *Finite Element Methods for Maxwell's Equations*. Oxford University Press, 2003.

[13] Vincent P E, Castonguay P, Jameson A. A New Class of High-Order Energy Stable Flux Reconstruction Schemes. *Journal of Scientific Computing*, 2011, 47:50–72. DOI:10.1007/s10915-010-9420-z.

[14] Vincent P E, Castonguay P, Jameson A. Insights from von Neumann analysis of high-order flux reconstruction schemes. *Journal of Computational Physics*, 2011, 230:8134–8154. DOI:10.1016/j.jcp.2011.07.013.

[15] Dumbser M, Zanotti O, Hidalgo A, Balsara D S. ADER-WENO Finite Volume Schemes with Space-Time Adaptive Mesh Refinement. https://arxiv.org/abs/1212.3585.

[16] Zennaro M. Natural continuous extensions of Runge–Kutta methods. *Mathematics of Computation*, 1986, 46. Author's institutional record: https://arts.units.it/handle/11368/1703220.

[17] Kennedy C A, Carpenter M H. *Additive Runge–Kutta Schemes for Convection–Diffusion–Reaction Equations*. NASA/TM-2001-211038, 2001. https://ntrs.nasa.gov/api/citations/20010075154/downloads/20010075154.pdf.

[18] NVIDIA. *CUDA Programming Guide*; *CUDA C++ Best Practices Guide*. https://docs.nvidia.com/cuda/cuda-programming-guide/; https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html. Accessed: 2026-11-10.

[19] MPI Forum. *MPI: A Message-Passing Interface Standard, Version 4.1*. 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf.

[20] PETSc. *KSP: Linear System Solvers*; *KSPFGMRES*. https://petsc.org/release/manual/ksp/; https://petsc.org/release/manualpages/KSP/KSPFGMRES/. Accessed: 2026-11-10.

[21] SciPy. *Sparse Linear Algebra Reference*. https://docs.scipy.org/doc/scipy/reference/sparse.linalg.html. Experiment version: 1.15.3; GMRES callback and stopping semantics follow this version and the executed checks.